

Manual Tecnico - EDDMail

Nombre: Angel Emanuel Rodriguez Corado

Carnet: 202404856

Arbol de EDDMail	2
UNIDADES	6
SLL (Lista Simple Usuarios)	6
DLL_CON (Lista Doblemente Enlazada Correos Recibidos)	15
UContactTypes (Lista Circular de Contactos)	18
UPila (Pila Correos Eliminados)	22
UCola (Cola de Correos Programados)	25
UMatriz (Matriz Dispersa de Relaciones)	28
Comunidades (Lista de Listas)	31
UAVL (Arbol AVL)	37
UBTREE (Arbol B de Orden 5)	45
BSTComunidades (Arbol BST)	54
Formularios	60
project1.lpr	60
uAbrirMenuRoot	61
uActualizarPerfil	67
uBandejaEntrada	69
uContactos	74
uCorreosProgramados	76
uCrearComunidades	78
uEnviarCorreo	81
uLogin	83
uPapelera	86
uPrincipal	88
uProgramarCorreo	93
uRegister	97
uReportes	99
uVerContactos	103
uBorradores	106
uFavoritos	112
uCrearComunidadesArbol	117

Arbol de EDDMail

```
.
├── Formularios
│   ├── backup
│   │   ├── project1.lpi
│   │   ├── project1.lpr
│   │   ├── project1.lps
│   │   ├── uabirmenuroot.lfm
│   │   ├── uabirmenuroot.pas
│   │   ├── uactualizarperfil.lfm
│   │   ├── uactualizarperfil.pas
│   │   ├── ubandejaentrada.lfm
│   │   ├── ubandejaentrada.pas
│   │   ├── ucontactos.lfm
│   │   ├── ucontactos.pas
│   │   ├── ucorreosprogramados.lfm
│   │   ├── ucrearcomunidades.lfm
│   │   ├── ucrearcomunidades.pas
│   │   ├── uenviarcorreo.lfm
│   │   ├── uenviarcorreo.pas
│   │   ├── ulogin.lfm
│   │   ├── ulogin.pas
│   │   ├── upapelera.lfm
│   │   ├── uprincipal.lfm
│   │   ├── uprincipal.pas
│   │   ├── uprogramarcorreo.lfm
│   │   ├── uprogramarcorreo.pas
│   │   ├── uregister.lfm
│   │   ├── uregister.pas
│   │   ├── ureportes.lfm
│   │   ├── ureportes.pas
│   │   ├── uvercontactos.lfm
│   │   └── uvercontactos.pas
│   ├── comunidades.dot
│   ├── comunidades.png
│   ├── lib
│   │   └── x86_64-linux
│   │       ├── comunidades.o
│   │       ├── comunidades.ppu
│   │       ├── dll_con.o
│   │       └── dll_con.ppu
```

```
— project1.compiled
— project1.o
— project1.or
— project1.res
— s11.o
— s11.ppu
— uabirmenuroot.lfm
— uabirmenuroot.o
— uabirmenuroot.ppu
— uactualizarperfil.lfm
— uactualizarperfil.o
— uactualizarperfil.ppu
— ubandejaentrada.lfm
— ubandejaentrada.o
— ubandejaentrada.ppu
— ucola.o
— ucola.ppu
— ucontactos.lfm
— ucontactos.o
— ucontactos.ppu
— ucontacttypes.o
— ucontacttypes.ppu
— ucorreosprogramados.lfm
— ucorreosprogramados.o
— ucorreosprogramados.ppu
— ucrearcomunidades.lfm
— ucrearcomunidades.o
— ucrearcomunidades.ppu
— uenviarcorreo.lfm
— uenviarcorreo.o
— uenviarcorreo.ppu
— ulogin.lfm
— ulogin.o
— ulogin.ppu
— umatriz.o
— umatriz.ppu
— upapelera.lfm
— upapelera.o
— upapelera.ppu
— upila.o
— upila.ppu
— uprincipal.lfm
— uprincipal.o
```



```
|— uprincipal.lfm
|— uprincipal.pas
|— uprogramarcorreio.lfm
|— uprogramarcorreio.pas
|— uregister.lfm
|— uregister.pas
|— ureportes.lfm
|— ureportes.pas
|— uvercontactos.lfm
|— uvercontactos.pas
└─ Unidades
    |— backup
    |   |— comunidades.pas
    |   |— dll_con.pas
    |   |— sll.pas
    |   |— ucola.pas
    |   |— ucontacttypes.pas
    |   |— umatriz.pas
    |   └─ upila.pas
    |— comunidades.pas
    |— dll_con.pas
    |— sll.pas
    |— ucola.pas
    |— ucontacttypes.pas
    |— umatriz.pas
    └─ upila.pas
```

8 directories, 144 files

UNIDADES

SLL (Lista Simple Usuarios)

```
unit SLL;
{$MODE DELPHI}

interface
uses
    DLL_CON, UContactTypes, UPila, UCola, Classes, SysUtils, Forms,
    Controls, Graphics, Dialogs, StdCtrls, Process;
type
    TDataUser = record
        id: string;
        name: string;
        email: string;
        user: string;
        phone: string;
        password: string;
        Contacts: PContactList;
        Inbox: PMsgList;
        Papelera: PPila;
        Programados: PCola;
    end;

    function SLL_INSERT(id, name, email, username, phone, password: string):
Boolean;
    procedure SSL_PRINT;
    function ValidatePassAndEmail(email, password: string): Boolean;
    function ValidatePassAndEmailLogin(email, password: string): Boolean;
    function SSL_GETBYEMAIL(const email: string): TDataUser;
    function SSL_GENERATE_DOT: string;
    function SLL_actualizarPerfil(email, username, phone: string): Boolean;
    procedure GenerateDotUsuarios(const DotFile, PNGFile: string);

implementation

type
    PNode = ^TNode;
    TNode = record
        id: string;
```

```

        name: string;
        email: string;
        username: string;
        phone: string;
        password: string;
        Next: PNode;
        Contacts: PContactList;
        Inbox: PMsgList;
        Papelera: PPila;
        Programados: PCola;

    end;

var
    Head: PNode = nil;

function SSL_GETBYEMAIL(const email: string): TDataUser;
var
    actual : PNode;

begin
    Result.id := '';
    Result.name := '';
    Result.email := '';
    Result.phone := '';
    Result.password := '';
    Result.user := '';

    if Head = nil then exit;

    actual := Head;
    while actual <> nil do
    begin
        if actual^.email = Trim(email) then
        begin
            Result.id := actual^.id;
            Result.user := actual^.username;
            Result.name := actual^.name;

```



```

        Result.email := actual^.email;
        Result.phone := actual^.phone;
        Result.password := actual^.password;
        Result.Contacts := actual^.Contacts;
        Result.Inbox := actual^.Inbox;
        Result.Papelera := actual^.Papelera;
        Result.Programados := actual^.Programados;

        Exit;
    end;
    actual := actual^.Next;
end;

end;

function EscapeDotString(const S: string): string;
var
    Res: string;
    i: integer;
begin
    Res := '';
    for i := 1 to Length(S) do
        begin
            case S[i] of
                '"': Res := Res + '\"';
                '\': Res := Res + '\\';
                '|': Res := Res + '\|';
                '{': Res := Res + '\{';
                '}': Res := Res + '\}';
                #10: Res := Res + '\n';
                #13: Res := Res + '\n';
            else
                Res := Res + S[i];
            end;
        end;
    end;

    Result := Res;
end;

```

```

function SLL_INSERT(id,name,email,username,phone,password:string):
Boolean;
var
    NewNode, Actual: PNode;
    isValid: Boolean;

begin
    isValid := ValidatePassAndEmail(email, password);
    if isValid then
        begin
            Result:= False;
            exit;
        end
    else
        begin
            New(NewNode);

            NewNode^.id := Trim(ID);
            NewNode^.name := Trim(name);
            NewNode^.email := Trim(email);
            NewNode^.username := Trim(username);
            NewNode^.phone := Trim(phone);
            NewNode^.password := Trim(password);
            NewNode^.Next := nil;
            New(NewNode^.Contacts);
            NewNode^.Contacts^.Head := nil;
            NewNode^.Contacts^.Last := nil;

            New(NewNode^.Inbox);
            NewNode^.Inbox^.Head := nil;
            NewNode^.Inbox^.Last := nil;

            New(NewNode^.Papelera);
            NewNode^.Papelera^.Top := nil;

            New(NewNode^.Programados);
            NewNode^.Programados^.Top := nil;
            NewNode^.Programados^.Tail := nil;
        end
    end
end

```

```

        if Head = nil then
        begin
            Head := NewNode;
        end

        else
        begin
            Actual := Head;
            while Actual^.Next <> nil do
                Actual := Actual^.Next;
                Actual^.Next := NewNode;
            end;

            Result:= True;
            exit;
        end;
    end;

function ValidatePassAndEmail(email, password: string):Boolean;
var
    Actual: PNode;
begin
    Result := False;

    if Head = nil then exit;

    Actual := Head;
    while Actual <> nil do
    begin
        if (Actual^.email = email) or (Actual^.password = password)
then
            begin
                Result := True;
                exit;
            end;
            Actual:= Actual^.Next;
        end;
    end;

function ValidatePassAndEmailLogin(email, password: string):Boolean;
var
    Actual: PNode;

```

```

begin
    Result := False;

    if Head = nil then exit;

    Actual := Head;
    while Actual <> nil do
    begin
        if (Actual^.email = email) and (Actual^.password = password)
then
            begin
                Result := True;
                exit;
            end;
            Actual:= Actual^.Next;
        end;
    end;

function SLL_actualizarPerfil(email, username, phone: string):Boolean;
var
    Actual: PNode;

begin
    Result := False;

    if Head = nil then exit;

    Actual := Head;
    while Actual <> nil do
    begin
        if (Actual^.email = email) then
            begin
                Actual^.username := Trim(username);
                Actual^.phone := Trim(phone);
                Result := True;
                exit;
            end;
            Actual:= Actual^.Next;
        end;
    end;

procedure SSL_PRINT;
var

```

```

    Actual: PNode;
begin
    if Head = nil then
    begin
        Writeln('Lista vacia!!!')
    end;
    Actual := Head;

    while Actual <> nil do
    begin
        Writeln('ID: ', Actual^.id);
        Writeln('Nombre: ', Actual^.name);
        Writeln('Correo: ', Actual^.email);
        Writeln('Usuario: ', Actual^.username);
        Writeln('Telefono: ', Actual^.phone);
        Writeln('Contraseña: ', Actual^.password);
        Writeln('-----');
        Actual := Actual^.Next;
    end;
end;

function SSL_GENERATE_DOT: string;
var
    SL: TStringList;
    Actual: PNode;
    Counter: Integer;
    NodeName, NextName: string;
    ResultText: string;

begin
    SL := TStringList.Create;

    SL.Add('digraph ListaEnlazada {');
    SL.Add('    rankdir=LR;');
    SL.Add('    nodesep=0.5;');
    SL.Add('');
    SL.Add('    subgraph cluster_0 {');
    SL.Add('        label="Lista Simple enlazada - Usuarios";');
    SL.Add('        fontsize=14;');
    SL.Add('        color=black;');
    SL.Add('        style=filled;');
    SL.Add('        fillcolor=white;');
    SL.Add('        node [shape=record, style=filled,

```

```

fillcolor=lightblue;');
    SL.Add('');

    if Head = nil then
    begin
        SL.Add('      null [label="LISTAVACIA", shape=plaintext;');
    end

    else
    begin
        Counter := 0;
        Actual := Head;
        while Actual <> nil do
        begin
            NodeName := Format('nodo%d', [Counter]);
            SL.Add(Format('      %s [label="{%s \n %s \n %s \n %s \n
%s}";',
                [NodeName,
                EscapeDotString(Actual^.id),
                EscapeDotString(Actual^.name),
                EscapeDotString(Actual^.username),
                EscapeDotString(Actual^.email),
                EscapeDotString(Actual^.phone)
                ]));

            if Actual^.Next <> nil then
            begin
                NextName := Format('nodo%d', [Counter + 1]);
                SL.Add(Format('      %s -> %s;', [NodeName,
NextName]));
            end;

            Inc(Counter);
            Actual := Actual^.Next;
        end;
    end;

    SL.Add('  }');
    SL.Add('}');

    ResultText := SL.Text;
    SL.Free;

```

```

        Result := ResultText;

    end;

procedure GenerateDotUsuarios(const DotFile, PNGFile: string);
var
    SL: TStringList;
    DotContent: string;
    Output: AnsiString;
begin
    // Generar el contenido .dot
    DotContent := SSL_GENERATE_DOT;

    SL := TStringList.Create;
    try
        SL.Text := DotContent;
        SL.SaveToFile(DotFile);
    finally
        SL.Free;
    end;

    // Ejecutar Graphviz para generar PNG
    if FileExists('/usr/bin/dot') then
        RunCommand('/usr/bin/dot', ['-Tpng', DotFile, '-o', PNGFile], Output)
    else if FileExists('C:\Program Files\Graphviz\bin\dot.exe') then
        RunCommand('C:\Program Files\Graphviz\bin\dot.exe', ['-Tpng', DotFile,
'-o', PNGFile], Output)
    else
        ShowMessage('No se encontró Graphviz (dot). Instálalo o ajusta la
ruta.');
```

ShowMessage('Reporte de usuarios generado en: ' + PNGFile);
 end;
end.

DLL_CON (Lista Doblemente Enlazada Correos Recibidos)

```
unit DLL_CON;

{$MODE DELPHI}

interface

uses
  SysUtils, Classes, Dialogs, Process;

type
  PNodeMsg = ^TNodeMsg;
  TNodeMsg = record
    id: string;
    remitente: string;
    asunto: string;
    mensaje: string;
    fecha: string;
    estado: string; // L = leído, NL = no leído
    Prev, Next: PNodeMsg;
  end;

  PMsgList = ^TMsgList;
  TMsgList = record
    Head, Last: PNodeMsg;
  end;

  procedure DLL_Insert(var L: PMsgList; id, remitente, asunto, mensaje,
    fecha: string);
  function DLL_Search(var L: PMsgList; id: string): PNodeMsg;
  procedure DLL_Print(var L: PMsgList);

  // Nuevo método para generar DOT y PNG de la bandeja del usuario
  procedure GenerateDotBandeja(Inbox: PMsgList; const DotFile, PNGFile:
    string);

implementation

  procedure DLL_Insert(var L: PMsgList; id, remitente, asunto, mensaje,
    fecha: string);
  var
    nuevo: PNodeMsg;
```



```

begin
    New(nuevo);
    nuevo^.id := id;
    nuevo^.remitente := remitente;
    nuevo^.asunto := asunto;
    nuevo^.mensaje := mensaje;
    nuevo^.fecha := fecha;
    nuevo^.estado := 'NL';
    nuevo^.Prev := nil;
    nuevo^.Next := nil;

    if L^.Head = nil then
    begin
        L^.Head := nuevo;
        L^.Last := nuevo;
    end
    else
    begin
        nuevo^.Prev := L^.Last;
        L^.Last^.Next := nuevo;
        L^.Last := nuevo;
    end;
end;

function DLL_Search(var L: PMsgList; id: string): PNodeMsg;
var
    actual: PNodeMsg;
begin
    actual := L^.Head;
    while (actual <> nil) and (actual^.id <> id) do
        actual := actual^.Next;
    Result := actual;
end;

procedure DLL_Print(var L: PMsgList);
var
    actual: PNodeMsg;
begin
    actual := L^.Head;
    while actual <> nil do
    begin
        Writeln('ID: ', actual^.id, ' Asunto: ', actual^.asunto, ' Estado: ',
        actual^.estado);
    end;
end;

```

```

    actual := actual^.Next;
end;
end;

procedure GenerateDotBandeja(Inbox: PMsgList; const DotFile, PNGFile:
string);
var
    SL: TStringList;
    actual: PNodeMsg;
    Output: AnsiString;
begin
    SL := TStringList.Create;
    try
        SL.Add('digraph Bandeja {');
        SL.Add('    rankdir=LR;');
        SL.Add('    node [shape=record, style=filled, fillcolor=lightyellow];');
        SL.Add('    label="Lista Doblemente Enlazada";');
        SL.Add('    labelloc=t; fontsize=20;');
        SL.Add('');

        if (Inbox = nil) or (Inbox^.Head = nil) then
            SL.Add('    null [label="Bandeja vacía", shape=plaintext];')
        else
            begin
                actual := Inbox^.Head;
                while actual <> nil do
                    begin
                        SL.Add(Format('    "%s" [label="{ID: %s | Remitente: %s | Asunto: %s | Fecha: %s | Estado: %s}";',
                            [actual^.id, actual^.id, actual^.remitente, actual^.asunto,
                                actual^.fecha, actual^.estado]));

                        if actual^.Next <> nil then
                            SL.Add(Format('    "%s" -> "%s" [dir=both];', [actual^.id,
                                actual^.Next^.id]));

                        actual := actual^.Next;
                    end;
                end;

                SL.Add('}');
                SL.SaveToFile(DotFile);
            end;
        end;
    except
        SL.Free;
    end;
end;

```

```

    // Ejecuta Graphviz para generar PNG
    if FileExists('/usr/bin/dot') then
        RunCommand('/usr/bin/dot', ['-Tpng', DotFile, '-o', PNGFile],
Output);

        ShowMessage('Reporte de Correos Recibidos generado en: ' + PNGFile);
    finally
        SL.Free;
    end;
end;

end.

```

UContactTypes (Lista Circular de Contactos)

```

unit UContactTypes;

{$MODE DELPHI}

interface

type
    PContactNode = ^TContactNode;
    TContactNode = record
        email: string;
        name: string;
        user: string;    // nuevo campo: nombre de usuario
        phone: string;   // nuevo campo: teléfono
        Next: PContactNode;
    end;

    PContactList = ^TContactList;
    TContactList = record
        Head, Last: PContactNode;
    end;

procedure InsertContact(var L: PContactList; email, name, user, phone:
string);
function SearchContact(var L: PContactList; email: string): PContactNode;
procedure PrintContacts(var L: PContactList);
procedure GenerateDotContactos(var L: PContactList; const DotFile, PNGFile:

```

```

string);

implementation
uses SysUtils, Classes, Process, Dialogs;

procedure InsertContact(var L: PContactList; email, name, user, phone:
string);
var
    nuevo: PContactNode;
begin
    New(nuevo);
    nuevo^.email := email;
    nuevo^.name := name;
    nuevo^.user := user;
    nuevo^.phone := phone;

    if L^.Head = nil then
    begin
        L^.Head := nuevo;
        L^.Last := nuevo;
        nuevo^.Next := nuevo; // circular
    end
    else
    begin
        nuevo^.Next := L^.Head;
        L^.Last^.Next := nuevo;
        L^.Last := nuevo;
    end;
end;

function SearchContact(var L: PContactList; email: string): PContactNode;
var
    actual: PContactNode;
begin
    Result := nil;
    if L^.Head = nil then Exit;

    actual := L^.Head;
    repeat
        if actual^.email = email then
        begin
            Result := actual;
            Exit;
        end
    until actual^.Next = nil;
end;

```

```

        end;
        actual := actual^.Next;
    until actual = L^.Head;
end;

procedure PrintContacts(var L: PContactList);
var
    actual: PContactNode;
begin
    if L^.Head = nil then
    begin
        Writeln('No hay contactos');
        Exit;
    end;

    actual := L^.Head;
    repeat
        Writeln('Nombre: ', actual^.name);
        Writeln('Usuario: ', actual^.user);
        Writeln('Correo: ', actual^.email);
        Writeln('Teléfono: ', actual^.phone);
        Writeln('-----');
        actual := actual^.Next;
    until actual = L^.Head;
end;

procedure GenerateDotContactos(var L: PContactList; const DotFile, PNGFile:
string);
var
    SL: TStringList;
    Actual: PContactNode;
    Output: AnsiString;
begin
    if (L = nil) or (L^.Head = nil) then
    begin
        ShowMessage('El usuario no tiene contactos.');
```

Exit;

end;

SL := TStringList.Create;

try

SL.Add('digraph Contactos {');

SL.Add(' rankdir=LR;');

SL.Add(' label="Lista Circular";');

```

SL.Add(' labelloc="t";');
SL.Add(' fontsize=14;');
SL.Add(' node [shape=record, style=filled, fillcolor=lightgreen];');
SL.Add('');

Actual := L^.Head;
repeat
    SL.Add(Format(' "%s" [label="{Nombre: %s | Usuario: %s | Email: %s |
Tel: %s}"]';',
        [Actual^.email, Actual^.name, Actual^.user, Actual^.email,
Actual^.phone]));

    if Actual^.Next <> nil then
    begin
        SL.Add(Format(' "%s" -> "%s";', [Actual^.email,
Actual^.Next^.email]));
        SL.Add(Format(' "%s" <- "%s";', [Actual^.email,
Actual^.Next^.email]));
    end;

    Actual := Actual^.Next;
until Actual = L^.Head;

SL.Add('}');
SL.SaveToFile(DotFile);

if FileExists('/usr/bin/dot') then
    RunCommand('/usr/bin/dot', ['-Tpng', DotFile, '-o', PNGFile],
Output);

    ShowMessage('Reporte de Contactos generado en: ' + PNGFile);
finally
    SL.Free;
end;
end;

end.

```

UPila (Pila Correos Eliminados)

```
unit UPila;

{$MODE DELPHI}

interface

uses
  Classes, SysUtils, Process, Dialogs, DLL_CON; // DLL_CON para usar
  TNodeMsg y PMsgList

type
  PNodePila = ^TNodePila;
  TNodePila = record
    correo: PNodeMsg;
    Next: PNodePila;
  end;

  PPila = ^TPila;
  TPila = record
    Top: PNodePila;
  end;

// Operaciones básicas de la pila
procedure Push(var P: PPila; correo: PNodeMsg);
function Pop(var P: PPila): PNodeMsg;
procedure PrintStack(var P: PPila);

// Generación de DOT para Graphviz
procedure GenerateDotPapelera(Pila: PPila; const DotFile, PNGFile: string);

implementation

procedure Push(var P: PPila; correo: PNodeMsg);
var
  nuevo: PNodePila;
begin
  New(nuevo);
  nuevo^.correo := correo;
  nuevo^.Next := P^.Top;
  P^.Top := nuevo;
end;
```

```

function Pop(var P: PPila): PNodeMsg;
var
    temp: PNodePila;
begin
    if P^.Top = nil then
    begin
        Result := nil;
        Exit;
    end;

    temp := P^.Top;
    Result := temp^.correo;
    P^.Top := temp^.Next;
    Dispose(temp);
end;

procedure PrintStack(var P: PPila);
var
    actual: PNodePila;
begin
    actual := P^.Top;
    while actual <> nil do
    begin
        Writeln('Papeleras: ', actual^.correo^.asunto);
        actual := actual^.Next;
    end;
end;

procedure GenerateDotPapeleras(Pila: PPila; const DotFile, PNGFile: string);
var
    SL: TStringList;
    Actual: PNodePila;
    Output: AnsiString;
begin
    SL := TStringList.Create;
    try
        SL.Add('digraph Papeleras {');
        SL.Add('    rankdir=TB;'); // Vertical: Top -> Bottom
        SL.Add('    node [shape=record, style=filled, fillcolor=lightblue];');
        SL.Add('    label="Pila";');
        SL.Add('    labelloc=t; fontsize=20;');
        SL.Add('');
    end;
end;

```



```

if (Pila = nil) or (Pila^.Top = nil) then
    SL.Add('  null [label="Papelera vacía", shape=plaintext];')
else
begin
    Actual := Pila^.Top;
    while Actual <> nil do
    begin
        SL.Add(Format('  "%s" [label="{ID: %s | Remitente: %s | Asunto: %s | Estado: %s}";',
            [Actual^.correo^.id, Actual^.correo^.id,
            Actual^.correo^.remitente, Actual^.correo^.asunto,
            Actual^.correo^.estado]));
        if Actual^.Next <> nil then
            SL.Add(Format('  "%s" -> "%s";', [Actual^.correo^.id,
            Actual^.Next^.correo^.id]));
        Actual := Actual^.Next;
    end;
end;

SL.Add('}');
SL.SaveToFile(DotFile);

// Ejecuta Graphviz para generar PNG
if FileExists('/usr/bin/dot') then
    RunCommand('/usr/bin/dot', ['-Tpng', DotFile, '-o', PNGFile],
Output);

    ShowMessage('Reporte de la Papelera generado en: ' + PNGFile);
finally
    SL.Free;
end;
end;

end.

```

UCola (Cola de Correos Programados)

```
unit UCola;

{$MODE DELPHI}

interface

uses DLL_CON, SysUtils, Classes, Process, Dialogs;

type
    PNodeCola = ^TNodeCola;
    TNodeCola = record
        correo: PNodeMsg;
        Next: PNodeCola;
    end;

    PCola = ^TCola;
    TCola = record
        Top, Tail: PNodeCola;
    end;

    procedure Enqueue(var C: PCola; correo: PNodeMsg);
    function Dequeue(var C: PCola): PNodeMsg;
    procedure PrintQueue(var C: PCola);
    procedure GenerateDotProgramados(Cola: PCola; const DotFile, PNGFile:
    string);

implementation

procedure Enqueue(var C: PCola; correo: PNodeMsg);
var nuevo: PNodeCola;
begin
    New(nuevo);
    nuevo^.correo := correo;
    nuevo^.Next := nil;

    if C^.Tail = nil then
    begin
        C^.Top := nuevo;
        C^.Tail := nuevo;
    end
    else
```

```

    begin
        C^.Tail^.Next := nuevo;
        C^.Tail := nuevo;
    end;
end;

function Dequeue(var C: PCola): PNodeMsg;
var temp: PNodeCola;
begin
    if C^.Top = nil then
    begin
        Result := nil;
        Exit;
    end;

    temp := C^.Top;
    Result := temp^.correo;
    C^.Top := temp^.Next;
    if C^.Top = nil then
        C^.Tail := nil;
    Dispose(temp);
end;

procedure PrintQueue(var C: PCola);
var actual: PNodeCola;
begin
    actual := C^.Top;
    while actual <> nil do
    begin
        Writeln('Correo programado: ', actual^.correo^.asunto);
        actual := actual^.Next;
    end;
end;

procedure GenerateDotProgramados(Cola: PCola; const DotFile, PNGFile:
string);
var
    SL: TStringList;
    Actual: PNodeCola;
    Output: AnsiString;
begin
    SL := TStringList.Create;
    try
        SL.Add('digraph Programados {');

```

```

SL.Add(' rankdir=LR;'); // Left to Right
SL.Add(' node [shape=record, style=filled, fillcolor=lightgreen];');
SL.Add(' label="Cola";');
SL.Add(' labelloc=t;'); // Título en la parte superior
SL.Add('');

if (Cola = nil) or (Cola^.Top = nil) then
    SL.Add(' null [label="Cola vacía", shape=plaintext];')
else
begin
    Actual := Cola^.Top;
    while Actual <> nil do
    begin
        SL.Add(Format(' "%s" [label="{ID: %s | Remitente: %s | Asunto: %s | Fecha: %s | Estado: %s}";',
            [Actual^.correo^.id, Actual^.correo^.id,
Actual^.correo^.remitente, Actual^.correo^.asunto, Actual^.correo^.fecha,
Actual^.correo^.estado]));
        if Actual^.Next <> nil then
            SL.Add(Format(' "%s" -> "%s";', [Actual^.correo^.id,
Actual^.Next^.correo^.id]));
        Actual := Actual^.Next;
    end;
end;

SL.Add('}');
SL.SaveToFile(DotFile);

// Ejecuta Graphviz para generar PNG
if FileExists('/usr/bin/dot') then
    RunCommand('/usr/bin/dot', ['-Tpng', DotFile, '-o', PNGFile],
Output);

    ShowMessage('Reporte de Correos Programados generado en: ' + PNGFile);
finally
    SL.Free;
end;
end;

end.

```

UMatriz (Matriz Dispersa de Relaciones)

```
unit UMatriz;

{$MODE DELPHI}

interface

uses
  SysUtils, Classes, Dialogs, Process;

type
  PMatrizNodo = ^TMatrizNodo;
  TMatrizNodo = record
    fila, columna: string; // email remitente y receptor
    cantidad: Integer;
    derecha, abajo: PMatrizNodo;
  end;

  PMatriz = ^TMatriz;
  TMatriz = record
    Head: PMatrizNodo;
  end;

procedure MatrizInit(var M: PMatriz);
procedure InsertRelacion(var M: PMatriz; fila, columna: string);
function SearchRelacion(var M: PMatriz; fila, columna: string):
  PMatrizNodo;
procedure PrintMatriz(var M: PMatriz);
procedure GenerateDotRelaciones(Matriz: PMatriz; const DotFile, PNGFile:
  string);

implementation

procedure MatrizInit(var M: PMatriz);
begin
  New(M);
  M^.Head := nil;
end;

procedure InsertRelacion(var M: PMatriz; fila, columna: string);
var
  Nodo: PMatrizNodo;
```

```

begin
  Nodo := SearchRelacion(M, fila, columna);
  if Nodo <> nil then
    begin
      Inc(Nodo^.cantidad);
      Exit;
    end;

    New(Nodo);
    Nodo^.fila := fila;
    Nodo^.columna := columna;
    Nodo^.cantidad := 1;
    Nodo^.derecha := M^.Head;
    Nodo^.abajo := nil;
    M^.Head := Nodo;
  end;

function SearchRelacion(var M: PMatriz; fila, columna: string):
PMatrizNodo;
var
  Actual: PMatrizNodo;
begin
  Actual := M^.Head;
  while Actual <> nil do
    begin
      if (Actual^.fila = fila) and (Actual^.columna = columna) then
        begin
          Result := Actual;
          Exit;
        end;
      Actual := Actual^.derecha;
    end;
  Result := nil;
end;

procedure PrintMatriz(var M: PMatriz);
var
  Actual: PMatrizNodo;
begin
  Actual := M^.Head;
  while Actual <> nil do
    begin
      Writeln(Actual^.fila, ' -> ', Actual^.columna, ' = ',

```

```

Actual^.cantidad);
    Actual := Actual^.derecha;
end;
end;

// -----
// Generación del DOT para Graphviz
procedure GenerateDotRelaciones(Matriz: PMatriz; const DotFile, PNGFile:
string);
var
    SL: TStringList;
    Actual: PMatrizNodo;
    Output: AnsiString;
begin
    if (Matriz = nil) or (Matriz^.Head = nil) then
    begin
        ShowMessage('La matriz de relaciones está vacía.');
```

Exit;

end;

SL := TStringList.Create;

try

SL.Add('digraph Relaciones {');

SL.Add(' rankdir=LR;'); // de izquierda a derecha

SL.Add(' node [shape=record, style=filled, fillcolor=lightyellow];');

SL.Add(' label="Matriz Dispersa de Relaciones";');

SL.Add(' labelloc=t;');

SL.Add('');

Actual := Matriz^.Head;

while Actual <> nil do

begin

SL.Add(Format(' "%s -> %s" [label="%d"];',

[Actual^.fila, Actual^.columna, Actual^.cantidad]));

Actual := Actual^.derecha;

end;

SL.Add('}');

SL.SaveToFile(DotFile);

// Ejecutar Graphviz si existe

if FileExists('/usr/bin/dot') then

RunCommand('/usr/bin/dot', ['-Tpng', DotFile, '-o', PNGFile],

```

Output);

    ShowMessage('Reporte de relaciones generado en: ' + PNGFile);
finally
    SL.Free;
end;
end;

end.

```

Comunidades (Lista de Listas)

```

unit Comunidades;

{$MODE DELPHI}

interface

uses
    Classes, SysUtils, Forms, Controls, Graphics, Dialogs, StdCtrls, SLL,
    Process;

type
    TUsuarioData = record
        correo: string;
    end;

    TComunidadData = record
        nombre: string;
    end;

function InsertarUsuario(nombreComunidad,email: string): Boolean;
function InsertarComunidad(nombre: string): Boolean;
function generateGrafComunidades(): string;

implementation

type
    PUsuarioCell = ^TUsuarioCell;
    TUsuarioCell = record
        correo: string;
    end;

```



```

    next: PUsuarioCell;
end;

PComunidadHeader = ^TComunidadHeader;
TComunidadHeader = record
    nombre: string;
    listaUsuarios: PUsuarioCell;
    next: PComunidadHeader;
end;

var
    Head: PComunidadHeader = nil;

function InsertarComunidad(nombre: string): Boolean;
var
    nuevaComunidad, actual: PComunidadHeader;
begin
    Result := False;
    New(nuevaComunidad);
    nuevaComunidad^.nombre := Trim(nombre);
    nuevaComunidad^.listaUsuarios := nil;
    nuevaComunidad^.next := nil;

    if Head = nil then
    begin
        Head := nuevaComunidad;
        Result := True;
        Exit;
    end;

    actual := Head;
    // Verificar si ya existe
    while actual <> nil do
    begin
        if SameText(actual^.nombre, nuevaComunidad^.nombre) then
        begin
            Dispose(nuevaComunidad);
            Exit;
        end;
        actual := actual^.next;
    end;

    // Insertar al final

```

```

actual := Head;
while actual^.next <> nil do
    actual := actual^.next;

actual^.next := nuevaComunidad;
Result := True;
end;

function InsertarUsuario(nombreComunidad, email: string): Boolean;
var
    comunidadActual: PComunidadHeader;
    nuevoUsuario, usuarioActual: PUsuarioCell;
    user: TDataUser;
begin
    Result := False;

    comunidadActual := Head;
    while (comunidadActual <> nil) and (comunidadActual^.nombre <>
Trim(nombreComunidad)) do
        comunidadActual := comunidadActual^.next;

    if comunidadActual = nil then Exit; // no encontrada

    // Verificar en SLL
    user := SSL_GETBYEMAIL(email);
    if user.email = '' then Exit;

    New(nuevoUsuario);
    nuevoUsuario^.correo := Trim(email);
    nuevoUsuario^.next := nil;

    if comunidadActual^.listaUsuarios = nil then
    begin
        comunidadActual^.listaUsuarios := nuevoUsuario;
        Result := True;
        Exit;
    end;

    usuarioActual := comunidadActual^.listaUsuarios;
    while usuarioActual <> nil do
    begin
        if SameText(usuarioActual^.correo, Trim(email)) then
            begin

```

```

        Dispose(nuevoUsuario);
        Exit;
    end;
    if usuarioActual^.next = nil then Break;
    usuarioActual := usuarioActual^.next;
end;

usuarioActual^.next := nuevoUsuario;
Result := True;
end;

function EscapeDotString(const S: string): string;
var
    Res: string;
    i: integer;
begin
    Res := '';
    for i := 1 to Length(S) do
        begin
            case S[i] of
                '"': Res := Res + '\"';
                '\': Res := Res + '\\';
                '|': Res := Res + '\|';
                '{': Res := Res + '\{';
                '}': Res := Res + '\}';
                #10, #13: Res := Res + '\n';
            else
                Res := Res + S[i];
            end;
        end;
    end;
    Result := Res;
end;

procedure GenerateDotComunidades(const DotFile, PNGFile: string);
var
    SL: TStringList;
    comunidadActual: PComunidadHeader;
    usuarioActual: PUsuarioCell;
    contadorComunidad, contadorUsuarios: Integer;
    nodoComunidad, nodoUsuario, nodoSig: string;
    Output: AnsiString;
begin
    SL := TStringList.Create;

```

```

try
  SL.Add('digraph Comunidades {');
  SL.Add('  rankdir=LR;');
  SL.Add('  nodesep=0.5;');
  SL.Add('  edge[arrowhead=normal];');
  SL.Add('  node [shape=box, style=filled, fillcolor=lightblue];');
  SL.Add('');

  if Head = nil then
    SL.Add('  null [label="LISTA VACÍA", shape=plaintext];')
  else
    begin
      contadorComunidad := 0;
      comunidadActual := Head;

      while comunidadActual <> nil do
        begin
          nodoComunidad := Format('comunidad%d', [contadorComunidad]);
          SL.Add(Format('  %s [label="%s", fillcolor=lightgreen];',
            [nodoComunidad, EscapeDotString(comunidadActual^.nombre)]));

          if comunidadActual^.next <> nil then
            begin
              nodoSig := Format('comunidad%d', [contadorComunidad + 1]);
              SL.Add(Format('  %s -> %s;', [nodoComunidad, nodoSig]));
            end;

          if comunidadActual^.listaUsuarios <> nil then
            begin
              contadorUsuarios := 0;
              usuarioActual := comunidadActual^.listaUsuarios;
              nodoUsuario := Format('usuario%d_%d', [contadorComunidad,
contadorUsuarios]);

              SL.Add(Format('  %s -> %s;', [nodoComunidad, nodoUsuario]));

              while usuarioActual <> nil do
                begin
                  nodoUsuario := Format('usuario%d_%d', [contadorComunidad,
contadorUsuarios]);
                  SL.Add(Format('  %s [label="%s"];',
                    [nodoUsuario, EscapeDotString(usuarioActual^.correo)]));

```

```

        if usuarioActual^.next <> nil then
        begin
            nodoSig := Format('usuario%d_%d', [contadorComunidad,
contadorUsuarios + 1]);
            SL.Add(Format(' %s -> %s;', [nodoUsuario, nodoSig]));
            end;

            Inc(contadorUsuarios);
            usuarioActual := usuarioActual^.next;
        end;

        SL.Add('{rank=same; ' + nodoComunidad);
        for contadorUsuarios := 0 to contadorUsuarios - 1 do
            SL.Add(' ' + Format('usuario%d_%d', [contadorComunidad,
contadorUsuarios]));
            SL.Add(' }');
        end;

        Inc(contadorComunidad);
        comunidadActual := comunidadActual^.next;
    end;
end;

SL.Add('}');
SL.SaveToFile(DotFile);

// Ejecuta Graphviz
if FileExists('/usr/bin/dot') then
    RunCommand('/usr/bin/dot', ['-Tpng', DotFile, '-o', PNGFile], Output)
else if FileExists('C:\Program Files\Graphviz\bin\dot.exe') then
    RunCommand('C:\Program Files\Graphviz\bin\dot.exe', ['-Tpng',
DotFile, '-o', PNGFile], Output)
else
    ShowMessage('No se encontró Graphviz (dot). Instálalo o ajusta la
ruta.');
```

```

    ShowMessage('Reporte de Comunidades generado en: ' + PNGFile);
finally
    SL.Free;
end;
end;

function generateGrafComunidades(): string;

```

```

var
  DotFile, PNGFile: string;
begin
  DotFile := 'comunidades.dot';
  PNGFile := 'comunidades.png';
  GenerateDotComunidades(DotFile, PNGFile);
  Result := PNGFile;
end;

end.

```

UAVL (Arbol AVL)

```

unit UAVL;

{$mode delphi}

interface

uses
  SysUtils;

type
  // Registro que representa un correo borrador
  TMail = record
    id: string;
    remitente: string;
    destinatario: string;
    asunto: string;
    mensaje: string;
  end;

  PNodeAVL = ^TNodeAVL;
  TNodeAVL = record
    mail: TMail;
    izquierda, derecha: PNodeAVL;
    altura: Integer;
  end;

  TAVLTree = record
    raiz: PNodeAVL;

```

```

end;
PAVLTree = ^TAVLTree;

function CrearAVL: PAVLTree;
function InsertarAVL(var raiz: PNodoAVL; correo: TMail): PNodoAVL;
function BuscarAVL(raiz: PNodoAVL; id: string): PNodoAVL;
procedure InOrdenAVL(nodo: PNodoAVL);
function EliminarAVL(var raiz: PNodoAVL; id: string): PNodoAVL;
procedure GenerarDOTAVL(raiz: PNodoAVL; const nombreArchivo: String);

implementation

// =====
// Funciones auxiliares
// =====

function GetAltura(nodo: PNodoAVL): Integer;
begin
    if nodo = nil then
        GetAltura := 0
    else
        GetAltura := nodo^.altura;
end;

procedure ActualizarAltura(nodo: PNodoAVL);
var
    alturaIzq, alturaDer: Integer;
begin
    alturaIzq := GetAltura(nodo^.izquierda);
    alturaDer := GetAltura(nodo^.derecha);
    if alturaIzq > alturaDer then
        nodo^.altura := 1 + alturaIzq
    else
        nodo^.altura := 1 + alturaDer;
end;

function GetBalance(nodo: PNodoAVL): Integer;
begin
    if nodo = nil then
        GetBalance := 0
    else
        GetBalance := GetAltura(nodo^.izquierda) - GetAltura(nodo^.derecha);
end;

```

```

// =====
// Rotaciones
// =====

function RotarDerecha(y: PNodeAVL): PNodeAVL;
var
    x: PNodeAVL;
begin
    x := y^.izquierda;
    y^.izquierda := x^.derecha;
    x^.derecha := y;
    ActualizarAltura(y);
    ActualizarAltura(x);
    RotarDerecha := x;
end;

function RotarIzquierda(x: PNodeAVL): PNodeAVL;
var
    y: PNodeAVL;
begin
    y := x^.derecha;
    x^.derecha := y^.izquierda;
    y^.izquierda := x;
    ActualizarAltura(x);
    ActualizarAltura(y);
    RotarIzquierda := y;
end;

// =====
// Crear nodo y árbol
// =====

function CrearNodo(correo: TMail): PNodeAVL;
var
    nuevo: PNodeAVL;
begin
    New(nuevo);
    nuevo^.mail := correo;
    nuevo^.izquierda := nil;
    nuevo^.derecha := nil;
    nuevo^.altura := 1;
    CrearNodo := nuevo;
end;

```



```

end;

function CrearAVL: PAVLTree;
var
    nuevo: PAVLTree;
begin
    New(nuevo);
    nuevo^.raiz := nil;
    CrearAVL := nuevo;
end;

// =====
// Insertar borrador
// =====

function InsertarAVL(var raiz: PNodeAVL; correo: TMail): PNodeAVL;
var
    balance: Integer;
begin
    if raiz = nil then
    begin
        raiz := CrearNodo(correo);
        Exit(raiz);
    end;

    if correo.id < raiz^.mail.id then
        raiz^.izquierda := InsertarAVL(raiz^.izquierda, correo)
    else if correo.id > raiz^.mail.id then
        raiz^.derecha := InsertarAVL(raiz^.derecha, correo)
    else
    begin
        raiz^.mail := correo; // si existe, actualiza
        Exit(raiz);
    end;

    ActualizarAltura(raiz);
    balance := GetBalance(raiz);

    if (balance > 1) and (correo.id < raiz^.izquierda^.mail.id) then
        Exit(RotarDerecha(raiz));

    if (balance < -1) and (correo.id > raiz^.derecha^.mail.id) then
        Exit(RotarIzquierda(raiz));

```

```

    if (balance > 1) and (correo.id > raiz^.izquierda^.mail.id) then
    begin
        raiz^.izquierda := RotarIzquierda(raiz^.izquierda);
        Exit(RotarDerecha(raiz));
    end;

    if (balance < -1) and (correo.id < raiz^.derecha^.mail.id) then
    begin
        raiz^.derecha := RotarDerecha(raiz^.derecha);
        Exit(RotarIzquierda(raiz));
    end;

    InsertarAVL := raiz;
end;

// =====
// Buscar borrador por ID
// =====

function BuscarAVL(raiz: PNodeAVL; id: string): PNodeAVL;
begin
    if (raiz = nil) or (raiz^.mail.id = id) then
        Exit(raiz);

    if id < raiz^.mail.id then
        Exit(BuscarAVL(raiz^.izquierda, id))
    else
        Exit(BuscarAVL(raiz^.derecha, id));
end;

// =====
// Recorrido in-orden
// =====

procedure InOrdenAVL(nodo: PNodeAVL);
begin
    if nodo <> nil then
    begin
        InOrdenAVL(nodo^.izquierda);
        Writeln('ID: ', nodo^.mail.id, ' | Asunto: ', nodo^.mail.asunto,
            ' | De: ', nodo^.mail.remitente, ' | Para: ',
            nodo^.mail.destinatario);
    end;
end;

```

```

        InOrdenAVL(nodo^.derecha);
    end;
end;

// =====
// Exportar a DOT
// =====

procedure EscribirLinea(var f: Text; const linea: String);
begin
    Writeln(f, linea);
end;

procedure GenerarNodosDOT(var f: Text; nodo: PNodeAVL);
begin
    if nodo = nil then Exit;

    EscribirLinea(f, Format(' "%s" [label="ID:%s\nAsunto:%s"];',
        [nodo^.mail.id, nodo^.mail.id, nodo^.mail.asunto]));

    if nodo^.izquierda <> nil then
    begin
        EscribirLinea(f, Format(' "%s" -> "%s";', [nodo^.mail.id,
nodo^.izquierda^.mail.id]));
        GenerarNodosDOT(f, nodo^.izquierda);
    end;

    if nodo^.derecha <> nil then
    begin
        EscribirLinea(f, Format(' "%s" -> "%s";', [nodo^.mail.id,
nodo^.derecha^.mail.id]));
        GenerarNodosDOT(f, nodo^.derecha);
    end;
end;

procedure GenerarDOTAVL(raiz: PNodeAVL; const nombreArchivo: String);
var
    f: Text;
begin
    Assign(f, nombreArchivo);
    Rewrite(f);

    EscribirLinea(f, 'digraph AVL {');

```

```

    EscribirLinea(f, ' node [shape=record, style=filled,
fillcolor=lightblue];');

    if raiz <> nil then
        GenerarNodosDOT(f, raiz)
    else
        EscribirLinea(f, ' vacio [label="(vacio)"];');

    EscribirLinea(f, '}');
    Close(f);
end;

function MinimoNodo(nodo: PNodoAVL): PNodoAVL;
begin
    while (nodo^.izquierda <> nil) do
        nodo := nodo^.izquierda;
    Result := nodo;
end;

function EliminarAVL(var raiz: PNodoAVL; id: string): PNodoAVL;
var
    temp: PNodoAVL;
    balance: Integer;
begin
    if raiz = nil then Exit(nil);

    if id < raiz^.mail.id then
        raiz^.izquierda := EliminarAVL(raiz^.izquierda, id)
    else if id > raiz^.mail.id then
        raiz^.derecha := EliminarAVL(raiz^.derecha, id)
    else
        begin
            // Nodo encontrado
            if (raiz^.izquierda = nil) or (raiz^.derecha = nil) then
                begin
                    if raiz^.izquierda <> nil then
                        temp := raiz^.izquierda
                    else
                        temp := raiz^.derecha;

                    if temp = nil then
                        begin
                            Dispose(raiz);

```

```

        raiz := nil;
        Exit(nil);
    end
else
begin
    raiz^ := temp^;
    Dispose(temp);
end;
end
else
begin
    temp := MinimoNodo(raiz^.derecha);
    raiz^.mail := temp^.mail;
    raiz^.derecha := EliminarAVL(raiz^.derecha, temp^.mail.id);
end;
end;

if raiz = nil then Exit(nil);

// Rebalancear
ActualizarAltura(raiz);
balance := GetBalance(raiz);

if (balance > 1) and (GetBalance(raiz^.izquierda) >= 0) then
    Exit(RotarDerecha(raiz));

if (balance > 1) and (GetBalance(raiz^.izquierda) < 0) then
begin
    raiz^.izquierda := RotarIzquierda(raiz^.izquierda);
    Exit(RotarDerecha(raiz));
end;

if (balance < -1) and (GetBalance(raiz^.derecha) <= 0) then
    Exit(RotarIzquierda(raiz));

if (balance < -1) and (GetBalance(raiz^.derecha) > 0) then
begin
    raiz^.derecha := RotarDerecha(raiz^.derecha);
    Exit(RotarIzquierda(raiz));
end;

Result := raiz;
end;

```

```
end.
```

UBTREE (Arbol B de Orden 5)

```
unit UBTree;

{$mode delphi}

interface

uses
  Classes, SysUtils;

const
  ORDEN = 5;
  MAX_CLAVES = ORDEN - 1; // 4
  MIN_CLAVES = ORDEN div 2 - 1; // 2

type
  // Registro para correos favoritos (tal como lo tenías)
  TMail = record
    id: string;
    remitente: string;
    destinatario: string;
    asunto: string;
    mensaje: string;
    // fecha: TDateTime; // si quieres fecha, agrégala aquí y ajústalo en
  uFavoritos
  end;

  // Nodo del Árbol B
  PNodeB = ^TNodeB;
  TNodeB = record
    n: Integer; // cantidad de claves
    claves: array[0..MAX_CLAVES-1] of string; // IDs de correos
    valores: array[0..MAX_CLAVES-1] of TMail; // correos completos
    hijos: array[0..ORDEN-1] of PNodeB; // hasta 5 hijos
    esHoja: Boolean;
```

```

end;

TBTTree = record
    raiz: PNodeB;
end;
PBTTree = ^TBTTree;

function CrearNodeB(esHoja: Boolean): PNodeB;
function CrearBTTree: PBTTree;
procedure InsertarB(var raiz: PNodeB; correo: TMail);
function BuscarB(nodo: PNodeB; id: string): TMail;
procedure InOrdenB(nodo: PNodeB);
procedure InOrdenBLista(nodo: PNodeB; lista: TStrings); // <-- agregado
procedure GenerarDOTB(raiz: PNodeB; const nombreArchivo: String);

procedure EliminarBTTree(var raiz: PNodeB; const id: string); // <--
agregado

implementation

// =====
// Crear nodo y árbol
// =====

function CrearNodeB(esHoja: Boolean): PNodeB;
var
    nuevo: PNodeB;
    i: Integer;
begin
    New(nuevo);
    nuevo^.n := 0;
    nuevo^.esHoja := esHoja;
    for i := 0 to ORDEN-1 do
        nuevo^.hijos[i] := nil;
    CrearNodeB := nuevo;
end;

function CrearBTTree: PBTTree;
var
    nuevo: PBTTree;
begin
    New(nuevo);
    nuevo^.raiz := nil;

```

```

    CrearBTree := nuevo;
end;

// =====
// Buscar (devuelve TMail; si no existe, devuelve registro con id = '')
// =====

function BuscarB(nodo: PNodeB; id: string): TMail;
var
    i: Integer;
    vacio: TMail;
begin
    vacio.id := '';
    if nodo = nil then Exit(vacio);

    i := 0;
    while (i < nodo^.n) and (id > nodo^.claves[i]) do
        Inc(i);

    if (i < nodo^.n) and (id = nodo^.claves[i]) then
        Exit(nodo^.valores[i]);

    if nodo^.esHoja then
        Exit(vacio)
    else
        Exit(BuscarB(nodo^.hijos[i], id));
end;

// =====
// Dividir hijo (cuando está lleno)
// =====

procedure DividirHijo(padre: PNodeB; i: Integer; hijo: PNodeB);
var
    nuevoNodo: PNodeB;
    j: Integer;
begin
    nuevoNodo := CrearNodeB(hijo^.esHoja);
    nuevoNodo^.n := MIN_CLAVES;

    for j := 0 to MIN_CLAVES-1 do
        begin
            nuevoNodo^.claves[j] := hijo^.claves[j + MIN_CLAVES + 1];

```



```

    nuevoNodo^.valores[j] := hijo^.valores[j + MIN_CLAVES + 1];
end;

if not hijo^.esHoja then
    for j := 0 to MIN_CLAVES do
        nuevoNodo^.hijos[j] := hijo^.hijos[j + MIN_CLAVES + 1];

hijo^.n := MIN_CLAVES;

for j := padre^.n downto i+1 do
    padre^.hijos[j+1] := padre^.hijos[j];

padre^.hijos[i+1] := nuevoNodo;

for j := padre^.n-1 downto i do
begin
    padre^.claves[j+1] := padre^.claves[j];
    padre^.valores[j+1] := padre^.valores[j];
end;

padre^.claves[i] := hijo^.claves[MIN_CLAVES];
padre^.valores[i] := hijo^.valores[MIN_CLAVES];
Inc(padre^.n);
end;

// =====
// Insertar en nodo no lleno
// =====

procedure InsertarNoLleno(nodo: PNodeB; correo: TMail);
var
    i: Integer;
begin
    i := nodo^.n - 1;

    if nodo^.esHoja then
    begin
        while (i >= 0) and (correo.id < nodo^.claves[i]) do
        begin
            nodo^.claves[i+1] := nodo^.claves[i];
            nodo^.valores[i+1] := nodo^.valores[i];
            Dec(i);
        end;
    end;
end;

```

```

    nodo^.claves[i+1] := correo.id;
    nodo^.valores[i+1] := correo;
    Inc(nodo^.n);
end
else
begin
    while (i >= 0) and (correo.id < nodo^.claves[i]) do
        Dec(i);
    Inc(i);

    if nodo^.hijos[i] = nil then
        nodo^.hijos[i] := CrearNodoB(True);

    if nodo^.hijos[i]^n = MAX_CLAVES then
        begin
            DividirHijo(nodo, i, nodo^.hijos[i]);
            if correo.id > nodo^.claves[i] then
                Inc(i);
            end;
            InsertarNoLleno(nodo^.hijos[i], correo);
        end;
    end;
end;

// =====
// Insertar en árbol
// =====

procedure InsertarB(var raiz: PNodeB; correo: TMail);
var
    nuevaRaiz: PNodeB;
begin
    if raiz = nil then
        begin
            raiz := CrearNodoB(True);
            raiz^.claves[0] := correo.id;
            raiz^.valores[0] := correo;
            raiz^.n := 1;
        end
    else
        begin
            if raiz^.n = MAX_CLAVES then
                begin
                    nuevaRaiz := CrearNodoB(False);

```

```

        nuevaRaiz^.hijos[0] := raiz;
        DividirHijo(nuevaRaiz, 0, raiz);
        InsertarNoLleno(nuevaRaiz, correo);
        raiz := nuevaRaiz;
    end
    else
        InsertarNoLleno(raiz, correo);
    end;
end;

// =====
// Recorrido in-orden (ruta de consola) - lo mantengo
// =====

procedure InOrdenB(nodo: PNodeB);
var
    i: Integer;
begin
    if nodo <> nil then
        begin
            for i := 0 to nodo^.n-1 do
                begin
                    if not nodo^.esHoja then
                        InOrdenB(nodo^.hijos[i]);
                    Writeln('ID: ', nodo^.claves[i], ' | Asunto: ',
nodo^.valores[i].asunto);
                end;
            if not nodo^.esHoja then
                InOrdenB(nodo^.hijos[nodo^.n]);
            end;
        end;
    end;

// =====
// Nuevo: Recorrido in-orden que llena un TStringList (para el grid)
// =====
procedure InOrdenBLista(nodo: PNodeB; lista: TStringList);
var
    i: Integer;
begin
    if (nodo = nil) or (lista = nil) then Exit;

    for i := 0 to nodo^.n - 1 do
        begin

```

```

    if not nodo^.esHoja then
        InOrdenBLista(nodo^.hijos[i], lista);

        // formato: ID|Asunto|Remitente
        lista.Add(nodo^.valores[i].id + '|' + nodo^.valores[i].asunto + '|' +
nodo^.valores[i].remitente);
    end;

    if not nodo^.esHoja then
        InOrdenBLista(nodo^.hijos[nodo^.n], lista);
end;

// =====
// Eliminar (versión simplificada: elimina claves si están en hojas)
// =====

procedure EliminarEnNodo(var nodo: PNodeB; const id: string);
var
    i, j: Integer;
begin
    if nodo = nil then Exit;

    // buscar posición
    i := 0;
    while (i < nodo^.n) and (id > nodo^.claves[i]) do Inc(i);

    if (i < nodo^.n) and (id = nodo^.claves[i]) then
    begin
        // encontrado en este nodo
        if nodo^.esHoja then
        begin
            // eliminar la entrada i (shift left)
            for j := i to nodo^.n - 2 do
            begin
                nodo^.claves[j] := nodo^.claves[j+1];
                nodo^.valores[j] := nodo^.valores[j+1];
            end;
            Dec(nodo^.n);
            Exit;
        end
        else
        begin
            // si está en nodo interno: versión simplificada -> intentar eliminar

```

```

en subárbol derecho
    // (una implementación completa requiere reemplazos y reequilibrio)
    EliminarEnNodo(nodo^.hijos[i+1], id);
    Exit;
end;
end
else
begin
    // no está en este nodo: bajar al hijo correspondiente si existe
    if nodo^.esHoja then
        Exit
    else
        begin
            if nodo^.hijos[i] <> nil then
                EliminarEnNodo(nodo^.hijos[i], id);
            end;
        end;
    end;
end;

procedure EliminarBTree(var raiz: PNodeB; const id: string);
var
    tmp: PNodeB;
begin
    if raiz = nil then Exit;

    EliminarEnNodo(raiz, id);

    // si la raiz quedó vacía (n = 0) y tiene hijos, promovemos el hijo 0
    como nueva raíz
    if (raiz <> nil) and (raiz^.n = 0) then
        begin
            if raiz^.esHoja then
                begin
                    Dispose(raiz);
                    raiz := nil;
                end
            else
                begin
                    tmp := raiz;
                    raiz := raiz^.hijos[0];
                    // opcional: liberar tmp (sin liberar sus hijos)
                    Dispose(tmp);
                end;
            end;
        end;
    end;
end;

```

```

    end;
end;

// =====
// Exportar a DOT
// =====

procedure EscribirLinea(var f: Text; const linea: String);
begin
    Writeln(f, linea);
end;

procedure GenerarNodosDOT(var f: Text; nodo: PNodeB; idNodo: Integer);
var
    i: Integer;
    clavesStr: String;
    idHijo: Integer;
begin
    if nodo = nil then Exit;

    clavesStr := '';
    for i := 0 to nodo^.n-1 do
        clavesStr := clavesStr + Format('|%s:%s', [nodo^.claves[i],
nodo^.valores[i].asunto]);

        EscribirLinea(f, Format('  nodo%d [label="%s", shape=record];', [idNodo,
clavesStr]));

        for i := 0 to nodo^.n do
            if nodo^.hijos[i] <> nil then
                begin
                    idHijo := idNodo * ORDEN + i + 1;
                    EscribirLinea(f, Format('  nodo%d -> nodo%d;', [idNodo, idHijo]));
                    GenerarNodosDOT(f, nodo^.hijos[i], idHijo);
                end;
            end;
        end;
    end;

procedure GenerarDOTB(raiz: PNodeB; const nombreArchivo: String);
var
    f: Text;
begin
    Assign(f, nombreArchivo);
    Rewrite(f);

```

```

    EscribirLinea(f, 'digraph ArbolB {');
    EscribirLinea(f, '  node [shape=record, style=filled,
fillcolor=lightblue];');

    if raiz <> nil then
        GenerarNodosDOT(f, raiz, 1)
    else
        EscribirLinea(f, '  vacio [label="(vacio)"];');

    EscribirLinea(f, '}');
    Close(f);
end;

end.

```

BSTComunidades (Arbol BST)

```

unit BSTCOMUNIDADES;

{$MODE DELPHI}

interface

uses
    SysUtils, Classes, SLL;

type
    TMsgData = record
        correo: string;
        mensaje: string;
        fecha: string;
    end;

    TComunidadData = record
        nombre: string;
    end;

    PUserData = record
        correo: string;
    end;

```

```

function InsertBSTNode(nombreComunidad, fecha: String): Boolean;
function insertUsers(nombreComunidad, email: string): Boolean;
function insertEmails(nombreComunidad, email, mensaje, fecha: string):
Boolean;
function BST_GENERARDOT(): string;

implementation

type
  PMsgCell = ^TMsgCell;
  TMsgCell = record
    correo: string;
    mensaje: string;
    fecha: string;
    next: PMsgCell;
  end;

  PUserCell = ^TUserCell;
  TUserCell = record
    correo: string;
    next: PUserCell;
  end;

  PComunidadHeader = ^TComunidadHeader;
  TComunidadHeader = record
    nombreComunidad: string;
    fecha: string;
    numeroMensajes: Integer;
    users: PUserCell;
    msgs: PMsgCell;
    left, right: PComunidadHeader;
  end;

var
  root: PComunidadHeader = nil;

function InsertBST(var root: PComunidadHeader; nombreComunidad, fecha:
String): Boolean;
var
  newCom: PComunidadHeader;
begin
  Result := False;

```



```

if root = nil then
begin
    New(newCom);
    newCom^.nombreComunidad := nombreComunidad;
    newCom^.fecha := fecha;
    newCom^.numeroMensajes := 0;
    newCom^.msgs := nil;
    newCom^.users := nil;
    newCom^.left := nil;
    newCom^.right := nil;

    root := newCom;
    Result := True;
end
else if nombreComunidad < root^.nombreComunidad then
    Result := InsertBST(root^.left, nombreComunidad, fecha)
else if nombreComunidad > root^.nombreComunidad then
    Result := InsertBST(root^.right, nombreComunidad, fecha)
else
    Exit; // ya existe
end;

function InsertBSTNode(nombreComunidad, fecha: String): Boolean;
begin
    Result := InsertBST(root, nombreComunidad, fecha);
end;

function SearchBST(var root: PComunidadHeader; nameCom: String):
PComunidadHeader;
begin
    if (root = nil) or (root^.nombreComunidad = nameCom) then
        Result := root
    else if nameCom < root^.nombreComunidad then
        Result := SearchBST(root^.left, nameCom)
    else
        Result := SearchBST(root^.right, nameCom);
end;

function insertUsers(nombreComunidad, email: string): Boolean;
var
    currentCom: PComunidadHeader;
    newUser, currentUser: PUserCell;
    user: TDataUser;

```

```

begin
    currentCom := SearchBST(root, nombreComunidad);
    user := SSL_GETBYEMAIL(email);

    if user.email = '' then
    begin
        Result := False;
        Exit;
    end;

    if currentCom <> nil then
    begin
        New(newUser);
        newUser^.correo := Trim(email);
        newUser^.next := nil;

        if currentCom^.users = nil then
        begin
            currentCom^.users := newUser;
            Result := True;
            Exit;
        end
        else
        begin
            currentUser := currentCom^.users;
            while (currentUser <> nil) do
            begin
                if SameText(currentUser^.correo, Trim(email)) then
                begin
                    Dispose(newUser);
                    Result := False;
                    Exit;
                end;
                if currentUser^.next = nil then
                begin
                    currentUser^.next := newUser;
                    Result := True;
                    Exit;
                end;
                currentUser := currentUser^.next;
            end;
        end;
    end
end

```

```

else
begin
    Result := False;
    Exit;
end;
end;

function insertEmails(nombreComunidad, email, mensaje, fecha: string):
Boolean;
var
    currentCom: PComunidadHeader;
    newMsg, currentMsg: PMsgCell;
    currentUser: PUserCell;
begin
    Result := False;
    currentCom := SearchBST(root, nombreComunidad);

    if currentCom <> nil then
    begin
        if currentCom^.users = nil then
        begin
            Result := False;
            Exit;
        end
        else
        begin
            currentUser := currentCom^.users;
            while (currentUser <> nil) do
            begin
                if SameText(currentUser^.correo, Trim(email)) then
                begin
                    New(newMsg);
                    newMsg^.correo := Trim(email);
                    newMsg^.mensaje := Trim(mensaje);
                    newMsg^.fecha := Trim(fecha);
                    newMsg^.next := nil;

                    if currentCom^.msgs = nil then
                    begin
                        currentCom^.msgs := newMsg;
                        currentCom^.numeroMensajes := 1;
                        Result := True;
                        Exit;
                    end
                end
            end
        end
    end
end;

```

```

        end
        else
        begin
            currentMsg := currentCom^.msgs;
            while (currentMsg^.next <> nil) do
                currentMsg := currentMsg^.next;
                Inc(currentCom^.numeroMensajes);
                currentMsg^.next := newMsg;
                Result := True;
                Exit;
            end;
        end;
        currentUser := currentUser^.next;
    end;
end;
end
else
begin
    Result := False;
    Exit;
end;
end;

procedure GenerarNodosDOT(var SL: TStringList; nodo: PComunidadHeader);
begin
    if nodo = nil then Exit;

    SL.Add(Format(
        '"%s" [label="Comunidad: %s\nFecha: %s\nMensajes: %d", shape=box,
style=filled, fillcolor=lightgreen];',
        [nodo^.nombreComunidad, nodo^.nombreComunidad, nodo^.fecha,
nodo^.numeroMensajes]
    ));

    if nodo^.left <> nil then
    begin
        SL.Add(Format('"%s" -> "%s";', [nodo^.nombreComunidad,
nodo^.left^.nombreComunidad]));
        GenerarNodosDOT(SL, nodo^.left);
    end;

    if nodo^.right <> nil then
    begin

```

```

        SL.Add(Format('%s' -> '%s';', [nodo^.nombreComunidad,
nodo^.right^.nombreComunidad]));
        GenerarNodosDOT(SL, nodo^.right);
    end;
end;

function BST_GENERARDOT: string;
var
    SL: TStringList;
begin
    SL := TStringList.Create;
    try
        SL.Add('digraph Comunidades {');
        SL.Add('    rankdir=TB;');
        SL.Add('    node [fontname="Arial", style=filled,
fillcolor=lightyellow];');

        if root <> nil then
            GenerarNodosDOT(SL, root)
        else
            SL.Add('"vacio" [label="(No hay comunidades)"]');

        SL.Add('}');
        Result := SL.Text;
    finally
        SL.Free;
    end;
end;

end.

```

Formularios

project1.lpr

```

program project1;

{$mode objfpc}{$H+}

```

```

uses
    {$IFDEF UNIX}
    cthreads,
    {$ENDIF}
    {$IFDEF HASAMIGA}
    athreads,
    {$ENDIF}
    Interfaces, // this includes the LCL widgetset
    Forms, uLogin, SLL
    { you can add units after this };

{$R *.res}

begin
    RequireDerivedFormResource:=True;
    Application.Scaled:=True;
    {$PUSH}{$WARN 5044 OFF}
    Application.MainFormOnTaskbar:=True;
    {$POP}
    Application.Initialize;
    Application.CreateForm(TfrmLogin, frmLogin);
    Application.Run;
end.

```

uAbrirMenuRoot

se encarga de Abrir el Menu del Usuario Root

```

unit Comunidades;

{$MODE DELPHI}

interface

uses
    Classes, SysUtils, Forms, Controls, Graphics, Dialogs, StdCtrls, SLL,
    Process;

type
    TUsuarioData = record
        correo: string;
    end;
end;

```

```

TComunidadData = record
    nombre: string;
end;

function InsertarUsuario(nombreComunidad,email: string): Boolean;
function InsertarComunidad(nombre: string): Boolean;

function generateGrafComunidades(): string;

implementation

type
    PUsuarioCell = ^TUsuarioCell;
    TUsuarioCell = record
        correo: string;
        next: PUsuarioCell;
    end;

    PComunidadHeader = ^TComunidadHeader;
    TComunidadHeader = record
        nombre: string;
        listaUsuarios: PUsuarioCell;
        next: PComunidadHeader;
    end;

var
    Head: PComunidadHeader = nil;

function InsertarComunidad(nombre: string): Boolean;
var
    nuevaComunidad, actual: PComunidadHeader;
begin
    Result := False;
    New(nuevaComunidad);
    nuevaComunidad^.nombre := Trim(nombre);
    nuevaComunidad^.listaUsuarios := nil;
    nuevaComunidad^.next := nil;

    if Head = nil then
    begin
        Head := nuevaComunidad;
        Result := True;
    end;
end;

```

```

    Exit;
end;

actual := Head;
// Verificar si ya existe
while actual <> nil do
begin
    if SameText(actual^.nombre, nuevaComunidad^.nombre) then
    begin
        Dispose(nuevaComunidad);
        Exit;
    end;
    actual := actual^.next;
end;

// Insertar al final
actual := Head;
while actual^.next <> nil do
    actual := actual^.next;

actual^.next := nuevaComunidad;
Result := True;
end;

function InsertarUsuario(nombreComunidad, email: string): Boolean;
var
    comunidadActual: PComunidadHeader;
    nuevoUsuario, usuarioActual: PUsuarioCell;
    user: TDataUser;
begin
    Result := False;

    comunidadActual := Head;
    while (comunidadActual <> nil) and (comunidadActual^.nombre <>
Trim(nombreComunidad)) do
        comunidadActual := comunidadActual^.next;

    if comunidadActual = nil then Exit; // no encontrada

    // Verificar en SLL
    user := SSL_GETBYEMAIL(email);
    if user.email = '' then Exit;

```



```

New(nuevoUsuario);
nuevoUsuario^.correo := Trim(email);
nuevoUsuario^.next := nil;

if comunidadActual^.listaUsuarios = nil then
begin
    comunidadActual^.listaUsuarios := nuevoUsuario;
    Result := True;
    Exit;
end;

usuarioActual := comunidadActual^.listaUsuarios;
while usuarioActual <> nil do
begin
    if SameText(usuarioActual^.correo, Trim(email)) then
    begin
        Dispose(nuevoUsuario);
        Exit;
    end;
    if usuarioActual^.next = nil then Break;
    usuarioActual := usuarioActual^.next;
end;

usuarioActual^.next := nuevoUsuario;
Result := True;
end;

function EscapeDotString(const S: string): string;
var
    Res: string;
    i: integer;
begin
    Res := '';
    for i := 1 to Length(S) do
    begin
        case S[i] of
            '"': Res := Res + '\"';
            '\': Res := Res + '\\';
            '|': Res := Res + '\|';
            '{': Res := Res + '\{';
            '}': Res := Res + '\}';
            #10, #13: Res := Res + '\n';
        else

```

```

        Res := Res + S[i];
    end;
end;
Result := Res;
end;

procedure GenerateDotComunidades(const DotFile, PNGFile: string);
var
    SL: TStringList;
    comunidadActual: PComunidadHeader;
    usuarioActual: PUsuarioCell;
    contadorComunidad, contadorUsuarios: Integer;
    nodoComunidad, nodoUsuario, nodoSig: string;
    Output: AnsiString;
begin
    SL := TStringList.Create;
    try
        SL.Add('digraph Comunidades {');
        SL.Add('    rankdir=LR;');
        SL.Add('    nodesep=0.5;');
        SL.Add('    edge[arrowhead=normal];');
        SL.Add('    node [shape=box, style=filled, fillcolor=lightblue];');
        SL.Add('');

        if Head = nil then
            SL.Add('    null [label="LISTA VACÍA", shape=plaintext];')
        else
            begin
                contadorComunidad := 0;
                comunidadActual := Head;

                while comunidadActual <> nil do
                    begin
                        nodoComunidad := Format('comunidad%d', [contadorComunidad]);
                        SL.Add(Format('    %s [label="%s", fillcolor=lightgreen];',
                            [nodoComunidad, EscapeDotString(comunidadActual^.nombre)]));

                        if comunidadActual^.next <> nil then
                            begin
                                nodoSig := Format('comunidad%d', [contadorComunidad + 1]);
                                SL.Add(Format('    %s -> %s;', [nodoComunidad, nodoSig]));
                            end;
                        end;
                    end;
                end;
            end;
        end;
    except
        Output := 'Error: No se pudo generar el archivo de salida.';
    end;
    SL.Free;
end;

```

```

    if comunidadActual^.listaUsuarios <> nil then
    begin
        contadorUsuarios := 0;
        usuarioActual := comunidadActual^.listaUsuarios;
        nodoUsuario := Format('usuario%d_%d', [contadorComunidad,
contadorUsuarios]);

        SL.Add(Format('  %s -> %s;', [nodoComunidad, nodoUsuario]));

        while usuarioActual <> nil do
        begin
            nodoUsuario := Format('usuario%d_%d', [contadorComunidad,
contadorUsuarios]);
            SL.Add(Format('  %s [label="%s"];',
[nodoUsuario, EscapeDotString(usuarioActual^.correo)]));

            if usuarioActual^.next <> nil then
            begin
                nodoSig := Format('usuario%d_%d', [contadorComunidad,
contadorUsuarios + 1]);
                SL.Add(Format('  %s -> %s;', [nodoUsuario, nodoSig]));
            end;

            Inc(contadorUsuarios);
            usuarioActual := usuarioActual^.next;
        end;

        SL.Add('{rank=same; ' + nodoComunidad);
        for contadorUsuarios := 0 to contadorUsuarios - 1 do
            SL.Add(' ' + Format('usuario%d_%d', [contadorComunidad,
contadorUsuarios]));
            SL.Add(' }');
        end;

        Inc(contadorComunidad);
        comunidadActual := comunidadActual^.next;
    end;
end;

SL.Add('}');
SL.SaveToFile(DotFile);

// Ejecuta Graphviz

```

```

    if FileExists('/usr/bin/dot') then
        RunCommand('/usr/bin/dot', ['-Tpng', DotFile, '-o', PNGFile], Output)
    else if FileExists('C:\Program Files\Graphviz\bin\dot.exe') then
        RunCommand('C:\Program Files\Graphviz\bin\dot.exe', ['-Tpng',
DotFile, '-o', PNGFile], Output)
    else
        ShowMessage('No se encontró Graphviz (dot). Instálalo o ajusta la
ruta.');
```



```

        ShowMessage('Reporte de Comunidades generado en: ' + PNGFile);
    finally
        SL.Free;
    end;
end;

function generateGrafComunidades(): string;
var
    DotFile, PNGFile: string;
begin
    DotFile := 'comunidades.dot';
    PNGFile := 'comunidades.png';
    GenerateDotComunidades(DotFile, PNGFile);
    Result := PNGFile;
end;

end.
```

uActualizarPerfil

se encarga de actualizar el nombre de usuario o telefono

```

unit uActualizarPerfil;

{$mode objfpc}{$H+}

interface

uses
    Classes, SysUtils, Forms, Controls, Graphics, Dialogs, StdCtrls, SLL;

type
```

```

{ TfrmActualizarPerfil }

TfrmActualizarPerfil = class(TForm)
    btnActualizar: TButton;
    edtUsuario: TEdit;
    edtTelefono: TEdit;
    lblUsuario: TLabel;
    lblTelefono: TLabel;
    procedure btnActualizarClick(Sender: TObject);
    procedure FormShow(Sender: TObject);
private
    FEmail: string; // Email fijo del usuario logueado
public
    property UsuarioEmail: string read FEmail write FEmail;
end;

var
    frmActualizarPerfil: TfrmActualizarPerfil;

implementation

{$R *.lfm}

{ TfrmActualizarPerfil }

procedure TfrmActualizarPerfil.FormShow(Sender: TObject);
var
    Data: TDataUser;
begin
    // Recuperamos datos actuales para mostrarlos al usuario
    Data := SSL_GETBYEMAIL(FEmail);

    if Data.email <> '' then
    begin
        edtUsuario.Text := Data.user;
        edtTelefono.Text := Data.phone;
    end;
end;

procedure TfrmActualizarPerfil.btnActualizarClick(Sender: TObject);
begin
    if (edtUsuario.Text = '') or (edtTelefono.Text = '') then
    begin

```

```

    ShowMessage('Por favor, complete todos los campos.');
```

Exit;

```

end;

if SLL_actualizarPerfil(FEmail, edtUsuario.Text, edtTelefono.Text) then
    ShowMessage('Perfil actualizado correctamente.')
else
    ShowMessage('Error: no se pudo actualizar el perfil.');
```

end;

```

end.
```

uBandejaEntrada

Se muestran los correos recibidos del usuario, se marcan como leídos o no leídos, se pueden eliminar y ordenar ascendentemente por letra

```

unit uBandejaEntrada;

{$mode delphi}{$H+}

interface

uses
    Classes, SysUtils, Forms, Controls, Graphics, Dialogs, StdCtrls,
    ComCtrls,
    DLL_CON, SLL, UPila;

type
    { TfrmBandejaEntrada }
    TfrmBandejaEntrada = class(TForm)
        lvCorreos: TListView;
        btnOrdenar: TButton;
        btnEliminar: TButton;
        lblNoLeidos: TLabel;
        memoMensaje: TMemo;
        procedure FormCreate(Sender: TObject);
        procedure lvCorreosSelectItem(Sender: TObject; Item: TListItem;
            Selected: Boolean);
        procedure btnOrdenarClick(Sender: TObject);
```

```

    procedure btnEliminarClick(Sender: TObject);
private
    FListaInbox: PMsgList;
    FCorreoActual: PNodeMsg;
    FUsuarioEmail: string;
    procedure CargarCorreos;
    procedure MostrarCorreo;
    procedure ActualizarNoLeidos;
public
    procedure SetUsuarioEmail(const email: string);
end;

var
    frmBandejaEntrada: TfrmBandejaEntrada;

implementation

{$R *.lfm}

procedure TfrmBandejaEntrada.SetUsuarioEmail(const email: string);
begin
    FUsuarioEmail := email;
    FListaInbox := SSL_GETBYEMAIL(FUsuarioEmail).Inbox;
    CargarCorreos;
end;

procedure TfrmBandejaEntrada.FormCreate(Sender: TObject);
begin
    Caption := 'Bandeja de Entrada';
    lvCorreos.ViewStyle := vsReport;
    lvCorreos.Columns.Clear;
    lvCorreos.Columns.Add.Caption := 'Estado';
    lvCorreos.Columns.Add.Caption := 'Asunto';
    lvCorreos.Columns.Add.Caption := 'Remitente';
    lvCorreos.Columns[0].Width := 50;
    lvCorreos.Columns[1].Width := 200;
    lvCorreos.Columns[2].Width := 150;
    memoMensaje.Clear;
    lblNoLeidos.Caption := 'No Leídos: 0';
end;

procedure TfrmBandejaEntrada.CargarCorreos;
var

```

```

    actual: PNodeMsg;
    item: TListItem;
begin
    lvCorreos.Items.Clear;
    if (FListaInbox = nil) or (FListaInbox^.Head = nil) then Exit;

    actual := FListaInbox^.Head;
    while actual <> nil do
    begin
        item := lvCorreos.Items.Add;
        item.Caption := actual^.estado;
        item.SubItems.Add(actual^.asunto);
        item.SubItems.Add(actual^.remitente);
        item.Data := actual;
        actual := actual^.Next;
    end;
    ActualizarNoLeidos;
end;

procedure TfrmBandejaEntrada.MostrarCorreo;
begin
    if FCorreoActual = nil then
    begin
        memoMensaje.Clear;
        Exit;
    end;
    memoMensaje.Text := FCorreoActual^.mensaje;
    if FCorreoActual^.estado = 'NL' then
    begin
        FCorreoActual^.estado := 'L';
        lvCorreos.Items[lvCorreos.ItemIndex].Caption := 'L';
        ActualizarNoLeidos;
    end;
end;

procedure TfrmBandejaEntrada.lvCorreosSelectItem(Sender: TObject; Item:
TListItem; Selected: Boolean);
begin
    if not Selected then Exit;
    FCorreoActual := PNodeMsg(Item.Data);
    MostrarCorreo;
end;

```



```

procedure TfrmBandejaEntrada.ActualizarNoLeidos;
var
    count: Integer;
    actual: PNodeMsg;
begin
    count := 0;
    if (FListaInbox <> nil) then
    begin
        actual := FListaInbox^.Head;
        while actual <> nil do
        begin
            if actual^.estado = 'NL' then Inc(count);
            actual := actual^.Next;
        end;
    end;
    lblNoLeidos.Caption := 'No Leídos: ' + IntToStr(count);
end;

procedure TfrmBandejaEntrada.btnOrdenarClick(Sender: TObject);
var
    i, j: Integer;
    Correos: array of PNodeMsg;
    temp: PNodeMsg;
    actual: PNodeMsg; // ♦ declarar aquí
begin
    if (FListaInbox = nil) or (FListaInbox^.Head = nil) then Exit;

    // Pasar la lista a un arreglo para ordenar
    SetLength(Correos, 0);
    actual := FListaInbox^.Head;
    while actual <> nil do
    begin
        SetLength(Correos, Length(Correos)+1);
        Correos[High(Correos)] := actual;
        actual := actual^.Next;
    end;

    // Bubble sort por asunto (ascendente)
    for i := 0 to High(Correos)-1 do
        for j := 0 to High(Correos)-i-1 do
            if CompareText(Correos[j]^asunto, Correos[j+1]^asunto) > 0 then
            begin
                temp := Correos[j];

```

```

        Correos[j] := Correos[j+1];
        Correos[j+1] := temp;
    end;

    // Reconstruir la lista doble
    FListaInbox^.Head := Correos[0];
    FListaInbox^.Head^.Prev := nil;
    for i := 1 to High(Correos) do
    begin
        Correos[i]^Prev := Correos[i-1];
        Correos[i-1]^Next := Correos[i];
    end;
    Correos[High(Correos)]^.Next := nil;

    // Recargar lista visual
    CargarCorreos;
end;

procedure TfrmBandejaEntrada.btnEliminarClick(Sender: TObject);
var
    item: TListItem;
    nodo: PNodeMsg;
    usuario: TDataUser;
begin
    if lvCorreos.ItemIndex = -1 then Exit;
    item := lvCorreos.Items[lvCorreos.ItemIndex];
    nodo := PNodeMsg(item.Data);

    // Obtener datos del usuario
    usuario := SSL_GETBYEMAIL(FUsuarioEmail);

    // Agregar a la pila de Papelera
    if usuario.Papelera = nil then
        New(usuario.Papelera);
    Push(usuario.Papelera, nodo);

    // Remover del DLL (Inbox)
    if nodo^.Prev <> nil then nodo^.Prev^.Next := nodo^.Next;
    if nodo^.Next <> nil then nodo^.Next^.Prev := nodo^.Prev;
    if nodo = FListaInbox^.Head then FListaInbox^.Head := nodo^.Next;
    if nodo = FListaInbox^.Last then FListaInbox^.Last := nodo^.Prev;

```

```

    // No hacer Dispose, la pila mantiene la referencia

    CargarCorreos;
end;

end.

```

uContactos

Se encarga de manejar la lista circular de contactos de cada usuario

```

unit uContactos;

{$mode objfpc}{$H+}

interface

uses
    Classes, SysUtils, Forms, Controls, Graphics, Dialogs, StdCtrls, SLL,
    UContactTypes;

type
    { TfrmContactos }

    TfrmContactos = class(TForm)
        btnAgregarContacto: TButton;
        edtCorreo: TEdit;
        lblCorreo: TLabel;
        procedure btnAgregarContactoClick(Sender: TObject);
        procedure FormCreate(Sender: TObject);
    public
        UsuarioEmail: string; // Email del usuario actual
    end;

var
    frmContactos: TfrmContactos;

implementation

{$R *.lfm}

```

```

{ TfrmContactos }

procedure TfrmContactos.FormCreate(Sender: TObject);
begin
    Caption := 'Agregar Contacto';
    lblCorreo.Caption := 'Correo del Contacto: ';
    btnAgregarContacto.Caption := 'Agregar Contacto';
end;

procedure TfrmContactos.btnAgregarContactoClick(Sender: TObject);
var
    correo: string;
    usuarioActual, contactoExistente: TDataUser;
    contacto: PContactNode;
begin
    correo := Trim(edtCorreo.Text);

    if correo = '' then
    begin
        ShowMessage('Debe ingresar un correo válido. ');
        Exit;
    end;

    // Verificar que el correo exista en la lista de usuarios
    if not ValidatePassAndEmail(correo, '') then
    begin
        ShowMessage('El correo no existe en el sistema. ');
        Exit;
    end;

    usuarioActual := SSL_GETBYEMAIL(UsuarioEmail);

    // Verificar que no esté agregado previamente
    contacto := SearchContact(usuarioActual.Contacts, correo);
    if contacto <> nil then
    begin
        ShowMessage('El contacto ya existe en su lista. ');
        Exit;
    end;

    // Agregar contacto con todos los datos
    contactoExistente := SSL_GETBYEMAIL(correo);

```

```

        InsertContact(usuarioActual.Contacts,
                      contactoExistente.email,
                      contactoExistente.name,
                      contactoExistente.user,
                      contactoExistente.phone);

        ShowMessage('Contacto agregado correctamente.');
```

edtCorreo.Clear;

```

end;

end.
```

uCorreosProgramados

Se encarga de mostrar los correos que se programaron en uProgramarCorreo

```

unit uCorreosProgramados;

{$mode delphi}{$H+}

interface

uses
    Classes, SysUtils, Forms, Controls, Graphics, Dialogs, StdCtrls,
    ComCtrls,
    SLL, DLL_CON, UCola, UContactTypes;

type
    { TfrmCorreosProgramados }

    TfrmCorreosProgramados = class(TForm)
        lvProgramados: TListView;
        btnEnviarAhora: TButton;
        procedure FormCreate(Sender: TObject);
        procedure btnEnviarAhoraClick(Sender: TObject);
    private
        FUsuario: TDataUser;
        procedure CargarProgramados;
    public
        procedure SetUsuario(const usuario: TDataUser);
    end;
```

```

var
    frmCorreosProgramados: TfrmCorreosProgramados;

implementation

{$R *.lfm}

procedure TfrmCorreosProgramados.FormCreate(Sender: TObject);
begin
    Caption := 'Correos Programados';
    btnEnviarAhora.Caption := 'Enviar Ahora';
end;

procedure TfrmCorreosProgramados.SetUsuario(const usuario: TDataUser);
begin
    FUsuario := usuario;
    CargarProgramados;
end;

procedure TfrmCorreosProgramados.CargarProgramados;
var
    nodo: PNodeCola;
    item: TListItem;
begin
    lvProgramados.Items.Clear;
    if (FUsuario.Programados = nil) or (FUsuario.Programados^.Top = nil) then
        Exit;

    nodo := FUsuario.Programados^.Top;
    while nodo <> nil do
    begin
        item := lvProgramados.Items.Add;
        item.Caption := nodo^.correo^.remitente;
        item.SubItems.Add(nodo^.correo^.asunto);
        item.SubItems.Add(nodo^.correo^.fecha);
        nodo := nodo^.Next;
    end;
end;

procedure TfrmCorreosProgramados.btnEnviarAhoraClick(Sender: TObject);
var
    nodo, siguiente: PNodeCola;
    destinatario: TDataUser;

```

```

begin
  nodo := FUsuario.Programados^.Top;
  while nodo <> nil do
    begin
      siguiente := nodo^.Next;
      // Buscar usuario destino
      destinatario := SSL_GETBYEMAIL(nodo^.correo^.remitente);
      // Insertar en bandeja de entrada
      DLL_Insert(destinatario.Inbox, nodo^.correo^.id,
nodo^.correo^.remitente,
      nodo^.correo^.asunto, nodo^.correo^.mensaje, nodo^.correo^.fecha);
      // Sacar de la cola
      Dequeue(FUsuario.Programados);
      nodo := siguiente;
    end;
    CargarProgramados;
    ShowMessage('Correos enviados.');
```

```
end;
```

```
end.
```

uCrearComunidades

Con el usuario root, se encarga de crear comunidades y agregar a diferentes usuarios a estas por medio del email

```

unit uCrearComunidades;

{$mode delphi}

interface

uses
  Classes, SysUtils, Forms, Controls, Graphics, Dialogs, StdCtrls,
  Comunidades;

type
  { TfrmCrearComunidades }
  TfrmCrearComunidades = class(TForm)
    lblTitulo: TLabel;
    lblNombre: TLabel;
    lblComunidad: TLabel;
```

```

    lblCorreo: TLabel;
    txtNombre: TEdit;
    txtComunidad: TEdit;
    txtCorreo: TEdit;
    btnCrear: TButton;
    btnAgregar: TButton;
    btnCerrar: TButton;
    btnReporte: TButton; // ♦ Nuevo botón
    procedure btnCrearClick(Sender: TObject);
    procedure btnAgregarClick(Sender: TObject);
    procedure btnCerrarClick(Sender: TObject);
    procedure btnReporteClick(Sender: TObject); // ♦ Nuevo evento
  private
  public
  end;

var
  frmCrearComunidades: TfrmCrearComunidades;

implementation

{$R *.lfm}

procedure TfrmCrearComunidades.btnCrearClick(Sender: TObject);
var
  response: Boolean;
begin
  response := InsertarComunidad(txtNombre.Text);
  if response then
    ShowMessage('Comunidad creada correctamente.')
  else
    ShowMessage('Error: Ya existe una comunidad con ese nombre.');
```

```

end;

procedure TfrmCrearComunidades.btnAgregarClick(Sender: TObject);
var
  response: Boolean;
begin
  response := InsertarUsuario(txtComunidad.Text, txtCorreo.Text);
  if response then
    ShowMessage('Usuario agregado correctamente a la comunidad.')
  else
    ShowMessage('Error: Verifica que la comunidad o el usuario existan o
```



```

que el usuario ya esté en la comunidad.');
```

```

end;

procedure TfrmCrearComunidades.btnCerrarClick(Sender: TObject);
begin
    Close;
end;

procedure TfrmCrearComunidades.btnReporteClick(Sender: TObject);
var
    dotFile, pngFile: string;
    dotCode: string;
    SL: TStringList;
begin
    dotFile := 'reportes/comunidades.dot';
    pngFile := 'reportes/comunidades.png';

    // Generamos el DOT desde Comunidades
    dotCode := generateGrafComunidades();

    // Guardamos el DOT
    SL := TStringList.Create;
    try
        SL.Text := dotCode;
        if not DirectoryExists('reportes') then
            CreateDir('reportes');
        SL.SaveToFile(dotFile);
    finally
        SL.Free;
    end;

    // Ejecutamos Graphviz para crear el PNG
    if ExecuteProcess('dot', ['-Tpng', dotFile, '-o', pngFile], []) = 0 then
        ShowMessage('Reporte generado en: ' + pngFile)
    else
        ShowMessage('Error al generar el reporte de comunidades.');
```

```

end;

end.

```

uEnviarCorreo

Se encarga de enviar un correo, este lleva destinatario, asunto y mensaje

```
unit uEnviarCorreo;

{$mode delphi}{$H+}

interface

uses
  Classes, SysUtils, Forms, Controls, Graphics, Dialogs, StdCtrls,
  DLL_CON, SLL, UContactTypes;

type

  { TfrmEnviarCorreo }

  TfrmEnviarCorreo = class(TForm)
    edtDestinatario: TEdit;
    edtAsunto: TEdit;
    memoMensaje: TMemo;
    btnEnviar: TButton;
    procedure btnEnviarClick(Sender: TObject);
  private
    FUsuario: TDataUser;
    function EstaEnContactos(const email: string): Boolean;
  public
    procedure SetUsuarioEmail(const email: string);
  end;

var
  frmEnviarCorreo: TfrmEnviarCorreo;

implementation

{$R *.lfm}

procedure TfrmEnviarCorreo.SetUsuarioEmail(const email: string);
begin
  FUsuario := SSL_GETBYEMAIL(email);
end;

function TfrmEnviarCorreo.EstaEnContactos(const email: string): Boolean;
```

```

var
    actual: PContactNode;
begin
    Result := False;

    // Verificar que la lista de contactos exista y no esté vacía
    if (FUsuario.Contacts = nil) or (FUsuario.Contacts^.Head = nil) then
        Exit;

    actual := FUsuario.Contacts^.Head;
    repeat
        if actual^.email = Trim(email) then
            begin
                Result := True;
                Exit;
            end;
        actual := actual^.Next;
    until actual = FUsuario.Contacts^.Head; // por ser lista circular
end;

procedure TfrmEnviarCorreo.btnEnviarClick(Sender: TObject);
var
    destinatarioEmail, idCorreo: string;
    inboxDestinatario: PMsgList;
begin
    destinatarioEmail := Trim(edtDestinatario.Text);
    if destinatarioEmail = '' then
        begin
            ShowMessage('Debe ingresar un correo destinatario.');
```

Exit;

end;

if not EstaEnContactos(destinatarioEmail) then

begin

ShowMessage('El destinatario no está en sus contactos.');

Exit;

end;

// Obtener inbox del destinatario

inboxDestinatario := SSL_GETBYEMAIL(destinatarioEmail).Inbox;

if inboxDestinatario = nil then

begin

```

        ShowMessage('Error: no se pudo acceder a la bandeja de entrada del
destinatario.');
```

```

        Exit;
    end;

    // Generar ID de correo
    idCorreo := FormatDateTime('yyyymmddhhnnsszzz', Now);

    // Insertar correo en inbox
    DLL_Insert(inboxDestinatario,
                idCorreo,
                FUsuario.email,
                edtAsunto.Text,
                memoMensaje.Text,
                FormatDateTime('dd/mm/yyyy hh:nn:ss', Now));

    ShowMessage('Correo enviado correctamente.');
```

```

    Close;
end;

end.
```

uLogin

Se encarga de iniciar sesion ya sea como usuario root o como usuario normal

```

unit uLogin;

{$mode delphi}

interface

uses
    Classes, SysUtils, Forms, Controls, Graphics, Dialogs, StdCtrls,
    SLL, uRegister, uPrincipal, uAbrirMenuRoot;

type
    TfrmLogin = class(TForm)
        btnLogin: TButton;
        btnRegister: TButton;
        edtEmail: TEdit;
```

```

    edtPassword: TEdit;
    lblEmail: TLabel;
    lblPassword: TLabel;
    procedure btnLoginClick(Sender: TObject);
    procedure btnRegisterClick(Sender: TObject);
private
    procedure AbrirMenuRoot;
    procedure AbrirMenuUsuario(email: string);
public
end;

var
    frmLogin: TfrmLogin;

implementation

{$R *.lfm}

procedure TfrmLogin.btnLoginClick(Sender: TObject);
var
    email, password: string;
begin
    email := Trim(edtEmail.Text);
    password := Trim(edtPassword.Text);

    if (email = '') or (password = '') then
    begin
        ShowMessage('Debe ingresar Email y Password');
        Exit;
    end;

    // Validación ROOT
    if (email = 'root@edd.com') and (password = 'root123') then
    begin
        ShowMessage('Bienvenido ROOT');
        AbrirMenuRoot;
        Exit;
    end;

    // Validación usuario normal
    if ValidatePassAndEmailLogin(email, password) then
    begin
        ShowMessage('Bienvenido ' + email);
    end;
end;

```

```

        AbrirMenuUsuario(email);
    end
    else
        ShowMessage('Credenciales incorrectas');
    end;

procedure TfrmLogin.btnRegisterClick(Sender: TObject);
begin
    frmRegister := TfrmRegister.Create(Application);
    try
        frmRegister.ShowModal;
    finally
        frmRegister.Free;
    end;
end;

procedure TfrmLogin.AbrirMenuRoot;
begin
    frmAbrirMenuRoot := TfrmAbrirMenuRoot.Create(Application);
    try
        frmAbrirMenuRoot.ShowModal; // Abre el formulario de Root de manera
modal
    finally
        frmAbrirMenuRoot.Free; // Libera la memoria al cerrarlo
    end;
end;

procedure TfrmLogin.AbrirMenuUsuario(email: string);
begin
    Hide; // Oculta login
    frmPrincipal := TfrmPrincipal.Create(Application);
    try
        frmPrincipal.UsuarioEmail := email; // Pasa el email al formulario
principal
        frmPrincipal.ShowModal;
    finally
        frmPrincipal.Free;
        Show; // Muestra login al cerrar el menú principal
    end;
end;

end.

```

uPapelera

Se encarga de gestionar los correos que se eliminan desde uBandejaEntrada, se puede buscar un correo en específico por medio del buscador

```
unit uPapelera;

{$mode objfpc}{$H+}

interface

uses
  Classes, SysUtils, Forms, Controls, Graphics, Dialogs, StdCtrls,
  ComCtrls,
  DLL_CON, UPila, SLL;

type

  { TfrmPapelera }

  TfrmPapelera = class(TForm)
    lvPapelera: TListView;
    btnEliminar: TButton;
    btnBuscar: TButton;
    edtBuscar: TEdit;
    lblBuscar: TLabel;
    procedure btnBuscarClick(Sender: TObject);
    procedure btnEliminarClick(Sender: TObject);
    procedure FormCreate(Sender: TObject);
  private
    FPila: PPila;
    FUsuario: TDataUser;
    procedure CargarPapelera;
  public
    procedure SetUsuario(const usuario: TDataUser);
  end;

var
  frmPapelera: TfrmPapelera;

implementation

{$R *.lfm}
```

```

procedure TfrmPapelera.SetUsuario(const usuario: TDataUser);
begin
    FUsuario := usuario;
    FPila := FUsuario.Papelera;
    CargarPapelera;
end;

procedure TfrmPapelera.FormCreate(Sender: TObject);
begin
    Caption := 'Papelera';
    lvPapelera.ViewStyle := vsReport;
    lvPapelera.Columns.Add.Caption := 'ID';
    lvPapelera.Columns.Add.Caption := 'Remitente';
    lvPapelera.Columns.Add.Caption := 'Asunto';
    lvPapelera.Columns.Add.Caption := 'Fecha';
end;

procedure TfrmPapelera.CargarPapelera;
var
    actual: PNodePila;
    item: TListItem;
begin
    lvPapelera.Items.Clear;
    if (FPila = nil) or (FPila^.Top = nil) then Exit;

    actual := FPila^.Top;
    while actual <> nil do
    begin
        item := lvPapelera.Items.Add;
        item.Caption := actual^.correo^.id;
        item.SubItems.Add(actual^.correo^.remitente);
        item.SubItems.Add(actual^.correo^.asunto);
        item.SubItems.Add(actual^.correo^.fecha);
        actual := actual^.Next;
    end;
end;

procedure TfrmPapelera.btnBuscarClick(Sender: TObject);
var
    palabra: string;
    actual: PNodePila;
    item: TListItem;
begin

```



```

palabra := Trim(edtBuscar.Text);
lvPapelera.Items.Clear;
if (FPila = nil) or (FPila^.Top = nil) then Exit;

actual := FPila^.Top;
while actual <> nil do
begin
    if Pos(LowerCase(palabra), LowerCase(actual^.correo^.asunto)) > 0 then
    begin
        item := lvPapelera.Items.Add;
        item.Caption := actual^.correo^.id;
        item.SubItems.Add(actual^.correo^.remitente);
        item.SubItems.Add(actual^.correo^.asunto);
        item.SubItems.Add(actual^.correo^.fecha);
    end;
    actual := actual^.Next;
end;
end;

procedure TfrmPapelera.btnEliminarClick(Sender: TObject);
var
    correo: PNodeMsg;
begin
    if (FPila = nil) or (FPila^.Top = nil) then
    begin
        ShowMessage('No hay correos para eliminar');
        Exit;
    end;

    correo := Pop(FPila);
    ShowMessage('Correo "' + correo^.asunto + '" eliminado
permanentemente.');
```

CargarPapelera;

end;

end.

uPrincipal

Este solo se puede acceder desde el uLogin como usuario normal, se encarga de manejar el menu principal, interactua con todas las demas vistas del programa

```

unit uPrincipal;

{$mode objfpc}{$H+}

interface

uses
  Classes, SysUtils, Forms, Controls, Graphics, Dialogs, StdCtrls,
  uContactos, uVerContactos, uActualizarPerfil, uBandejaEntrada,
  uEnviarCorreo, uProgramarCorreo, uCorreosProgramados, uPapelera,
  uReportes, SLL;

type
  { TfrmPrincipal }

  TfrmPrincipal = class(TForm)
    btnContactos: TButton;
    btnVerContactos: TButton;
    btnActualizarPerfil: TButton;
    btnBandeja: TButton;
    btnEnviarCorreo: TButton;
    btnProgramarCorreo: TButton;
    btnCorreosProgramados: TButton;
    btnPapelera: TButton;
    btnReportes: TButton;
    lblUsuario: TLabel;
    procedure btnContactosClick(Sender: TObject);
    procedure btnVerContactosClick(Sender: TObject);
    procedure btnActualizarPerfilClick(Sender: TObject);
    procedure btnBandejaClick(Sender: TObject);
    procedure btnEnviarCorreoClick(Sender: TObject);
    procedure btnProgramarCorreoClick(Sender: TObject);
    procedure btnCorreosProgramadosClick(Sender: TObject);
    procedure btnPapeleraClick(Sender: TObject);
    procedure btnReportesClick(Sender: TObject); // <-- agregado
    procedure FormCreate(Sender: TObject);
  public
    UsuarioEmail: string; // Email del usuario actual
  end;

var
  frmPrincipal: TfrmPrincipal;
  frmReportes: TfrmReportes; // Declaración global del formulario de

```

reportes

implementation

{ \$R *.lfm }

```
procedure TfrmPrincipal.FormCreate(Sender: TObject);
begin
```

```
    Caption := 'EDDMail - Usuario';
    lblUsuario.Caption := 'Usuario: ' + UsuarioEmail;

    btnContactos.Caption := 'Agregar Contacto';
    btnVerContactos.Caption := 'Ver Contactos';
    btnActualizarPerfil.Caption := 'Actualizar Perfil';
    btnBandeja.Caption := 'Bandeja de Entrada';
    btnEnviarCorreo.Caption := 'Enviar Correo';
    btnProgramarCorreo.Caption := 'Programar Correo';
    btnCorreosProgramados.Caption := 'Correos Programados';
    btnReportes.Caption := 'Reportes';
    btnPapelera.Caption := 'Papelera';
end;
```

```
procedure TfrmPrincipal.btnContactosClick(Sender: TObject);
begin
```

```
    frmContactos := TfrmContactos.Create(Application);
    try
        frmContactos.UsuarioEmail := UsuarioEmail;
        frmContactos.ShowModal;
    finally
        frmContactos.Free;
    end;
end;
```

```
procedure TfrmPrincipal.btnVerContactosClick(Sender: TObject);
begin
```

```
    frmVerContactos := TfrmVerContactos.Create(Application);
    try
        frmVerContactos.UsuarioEmail := UsuarioEmail;
        frmVerContactos.InicializarContactos;
        frmVerContactos.ShowModal;
    finally
        frmVerContactos.Free;
    end;
```

```

end;

procedure TfrmPrincipal.btnActualizarPerfilClick(Sender: TObject);
begin
    frmActualizarPerfil := TfrmActualizarPerfil.Create(Application);
    try
        frmActualizarPerfil.UsuarioEmail := UsuarioEmail;
        frmActualizarPerfil.ShowModal;
    finally
        frmActualizarPerfil.Free;
    end;
end;

procedure TfrmPrincipal.btnBandejaClick(Sender: TObject);
begin
    frmBandejaEntrada := TfrmBandejaEntrada.Create(Application);
    try
        frmBandejaEntrada.SetUsuarioEmail(UsuarioEmail);
        frmBandejaEntrada.ShowModal;
    finally
        frmBandejaEntrada.Free;
    end;
end;

procedure TfrmPrincipal.btnEnviarCorreoClick(Sender: TObject);
begin
    frmEnviarCorreo := TfrmEnviarCorreo.Create(Application);
    try
        frmEnviarCorreo.SetUsuarioEmail(UsuarioEmail);
        frmEnviarCorreo.ShowModal;
    finally
        frmEnviarCorreo.Free;
    end;
end;

procedure TfrmPrincipal.btnProgramarCorreoClick(Sender: TObject);
var
    usuario: TDataUser;
begin
    usuario := SSL_GETBYEMAIL(UsuarioEmail);
    frmProgramarCorreo := TfrmProgramarCorreo.Create(Application);
    try
        frmProgramarCorreo.SetUsuario(usuario);
    end;
end;

```

```

        frmProgramarCorreo.ShowModal;
    finally
        frmProgramarCorreo.Free;
    end;
end;

procedure TfrmPrincipal.btnCorreosProgramadosClick(Sender: TObject);
var
    usuario: TDataUser;
begin
    usuario := SSL_GETBYEMAIL(UsuarioEmail);
    frmCorreosProgramados := TfrmCorreosProgramados.Create(Application);
    try
        frmCorreosProgramados.SetUsuario(usuario);
        frmCorreosProgramados.ShowModal;
    finally
        frmCorreosProgramados.Free;
    end;
end;

procedure TfrmPrincipal.btnPapeleraClick(Sender: TObject);
var
    usuario: TDataUser;
begin
    usuario := SSL_GETBYEMAIL(UsuarioEmail);
    frmPapelera := TfrmPapelera.Create(Application);
    try
        frmPapelera.SetUsuario(usuario);
        frmPapelera.ShowModal;
    finally
        frmPapelera.Free;
    end;
end;

procedure TfrmPrincipal.btnReportesClick(Sender: TObject);
var
    usuario: TDataUser;
begin
    // Obtiene la información del usuario actual
    usuario := SSL_GETBYEMAIL(UsuarioEmail);

    // Verifica que se haya encontrado el usuario
    if usuario.email = '' then

```

```

begin
    ShowMessage('Usuario no encontrado.');
```

```

    Exit;
end;

// Crear la instancia del formulario de reportes si no existe
if not Assigned(frmReportes) then
    frmReportes := TfrmReportes.Create(Application);

// Pasa el usuario actual al formulario de reportes
frmReportes.SetUsuario(usuario);

// Muestra el formulario de reportes como modal
frmReportes.ShowModal;
end;

end.
```

uProgramarCorreo

Este se encarga de programar un correo, el correo programado lleva los mismos componentes que un correo normal, pero agrega fecha y hora de envío

```

unit uProgramarCorreo;

{$mode objfpc}{$H+}

interface

uses
    Classes, SysUtils, Forms, Controls, Graphics, Dialogs, StdCtrls,
    SLL, UCola, DLL_CON, UContactTypes;

type

    { TfrmProgramarCorreo }

    TfrmProgramarCorreo = class(TForm)
        lblDestinatario: TLabel;
        lblAsunto: TLabel;
        lblMensaje: TLabel;
        lblFecha: TLabel;
```

```

    lblHora: TLabel;
    edtDestinatario: TEdit;
    edtAsunto: TEdit;
    memMensaje: TMemo;
    edtFecha: TEdit;
    edtHora: TEdit;
    btnProgramar: TButton;
    procedure btnProgramarClick(Sender: TObject);
private
    FUsuario: TDataUser;
    function EstaEnContactos(const email: string): Boolean;
public
    procedure SetUsuario(usuario: TDataUser);
end;

var
    frmProgramarCorreo: TfrmProgramarCorreo;

implementation

{$R *.lfm}

procedure TfrmProgramarCorreo.SetUsuario(usuario: TDataUser);
begin
    FUsuario := usuario;
end;

// Función para validar si un correo está en los contactos
function TfrmProgramarCorreo.EstaEnContactos(const email: string): Boolean;
var
    actual: PContactNode;
begin
    Result := False;
    if FUsuario.Contacts = nil then Exit;
    actual := FUsuario.Contacts^.Head;
    if actual = nil then Exit;

    repeat
        if actual^.email = Trim(email) then
            begin
                Result := True;
                Exit;
            end;
    end;
end;

```

```

    actual := actual^.Next;
until actual = FUsuario.Contacts^.Head;
end;

procedure TfrmProgramarCorreo.btnProgramarClick(Sender: TObject);
var
    Dia, Mes, Anio: Word;
    Hora, Minuto: Word;
    FechaStr, HoraStr, Destinatario: string;
    FechaProgramada: TDateTime;
    correo: PNodeMsg;
begin
    Destinatario := Trim(edtDestinatario.Text);

    // Validar destinatario
    if Destinatario = '' then
    begin
        ShowMessage('Debe ingresar un destinatario');
        Exit;
    end;

    if not EstaEnContactos(Destinatario) then
    begin
        ShowMessage('El destinatario no está en sus contactos');
        Exit;
    end;

    FechaStr := Trim(edtFecha.Text); // formato dd/mm/yyyy
    HoraStr := Trim(edtHora.Text);   // formato hh:nn

    // Validar fecha
    if Length(FechaStr) <> 10 then
    begin
        ShowMessage('Fecha inválida. Formato: dd/mm/yyyy');
        Exit;
    end;

    try
        Dia := StrToInt(Copy(FechaStr, 1, 2));
        Mes := StrToInt(Copy(FechaStr, 4, 2));
        Anio := StrToInt(Copy(FechaStr, 7, 4));
        FechaProgramada := EncodeDate(Anio, Mes, Dia);
    except

```



```

    ShowMessage('Fecha inválida. Formato: dd/mm/yyyy');
    Exit;
end;

// Validar hora
if Length(HoraStr) <> 5 then
begin
    ShowMessage('Hora inválida. Formato: hh:nn (24h)');
    Exit;
end;

try
    Hora := StrToInt(Copy(HoraStr, 1, 2));
    Minuto := StrToInt(Copy(HoraStr, 4, 2));
    if (Hora > 23) or (Minuto > 59) then
        raise Exception.Create('Hora fuera de rango');
    FechaProgramada := FechaProgramada + EncodeTime(Hora, Minuto, 0, 0);
except
    ShowMessage('Hora inválida. Formato: hh:nn (24h)');
    Exit;
end;

// Crear correo
New(correo);
correo^.id := IntToStr(Random(1000000));
correo^.remiteinte := FUsuario.email;
correo^.asunto := edtAsunto.Text;
correo^.mensaje := memMensaje.Text;
correo^.fecha := DateTimeToStr(FechaProgramada);
correo^.estado := 'NL';
correo^.Prev := nil;
correo^.Next := nil;

// Agregar destinatario como campo adicional si lo deseas
correo^.remiteinte := Destinatario; // o crear campo "destinatario" en
PNodeMsg si quieres

// Agregar a la cola del usuario
Enqueue(FUsuario.Programados, correo);

ShowMessage('Correo programado correctamente para ' +
FormatDateTime('dd/mm/yyyy hh:nn', FechaProgramada));

```

```

// Limpiar campos
edtDestinatario.Text := '';
edtAsunto.Text := '';
memMensaje.Lines.Clear;
edtFecha.Text := '';
edtHora.Text := '';
end;

end.

```

uRegister

Se encarga de agregar usuarios de forma manual, pide id, nombre, nombre usuario, email, telefono y contraseña

```

unit uRegister;

{$mode delphi}

interface

uses
  Classes, SysUtils, Forms, Controls, Graphics, Dialogs, StdCtrls,
  SLL; // lista simple de usuarios

type
  TfrmRegister = class(TForm)
    btnCancelar: TButton;
    btnRegistrar: TButton;
    edtID: TEdit;
    edtNombre: TEdit;
    edtEmail: TEdit;
    edtUsuario: TEdit;
    edtTelefono: TEdit;
    edtPassword: TEdit;
    lblID: TLabel;
    lblNombre: TLabel;
    lblEmail: TLabel;
    lblUsuario: TLabel;
    lblTelefono: TLabel;
    lblPassword: TLabel;
    procedure btnRegistrarClick(Sender: TObject);
  end;

```

```

    procedure btnCancelarClick(Sender: TObject);
private

public

end;

var
    frmRegister: TfrmRegister;

implementation

{$R *.lfm}

{ TfrmRegister }

procedure TfrmRegister.btnRegistrarClick(Sender: TObject);
var
    id, nombre, email, usuario, telefono, password: string;
begin
    id := Trim(edtID.Text);
    nombre := Trim(edtNombre.Text);
    email := Trim(edtEmail.Text);
    usuario := Trim(edtUsuario.Text);
    telefono := Trim(edtTelefono.Text);
    password := Trim(edtPassword.Text);

    if (id = '') or (nombre = '') or (email = '') or (usuario = '') or
        (telefono = '') or (password = '') then
    begin
        ShowMessage('Debe llenar todos los campos');
        Exit;
    end;

    if SLL_INSERT(id, nombre, email, usuario, telefono, password) then
    begin
        ShowMessage('Usuario registrado correctamente');
        Close; // cierra formulario y regresa al login
    end
    else
    begin
        ShowMessage('Error: el email o password ya existe');
    end;
end;

```

```

end;

procedure TfrmRegister.btnCancelarClick(Sender: TObject);
begin
    Close; // vuelve al login
end;

end.

```

uReportes

Se encarga de generar los diferentes reportes del usuario

```

unit uReportes;

{$mode objfpc}{$H+}

interface

uses
    Classes, SysUtils, Forms, Controls, Graphics, Dialogs, StdCtrls,
    SLL, DLL_CON, UPila, UCola, UContactTypes;

type
    TfrmReportes = class(TForm)
        btnReporteCorreos: TButton;
        btnReportePapelera: TButton;
        btnReporteProgramados: TButton;
        btnReporteContactos: TButton;
        btnCerrar: TButton;

        procedure btnCerrarClick(Sender: TObject);
        procedure btnReporteCorreosClick(Sender: TObject);
        procedure btnReportePapeleraClick(Sender: TObject);
        procedure btnReporteProgramadosClick(Sender: TObject);
        procedure btnReporteContactosClick(Sender: TObject);

    private
        FUsuario: TDataUser;

    public
        procedure SetUsuario(AUser: TDataUser);

```

```

    procedure GenerarReporteCorreos;
    procedure GenerarReportePapelera;
    procedure GenerarReporteProgramados;
    procedure GenerarReporteContactos;

end;

var
    frmReportes: TfrmReportes;

implementation

{$R *.lfm}

{ TfrmReportes }

procedure TfrmReportes.SetUsuario(AUser: TDataUser);
begin
    FUsuario := AUser;
end;

procedure TfrmReportes.btnCerrarClick(Sender: TObject);
begin
    Close;
end;

procedure TfrmReportes.btnReporteCorreosClick(Sender: TObject);
begin
    GenerarReporteCorreos;
end;

procedure TfrmReportes.btnReportePapeleraClick(Sender: TObject);
begin
    GenerarReportePapelera;
end;

procedure TfrmReportes.btnReporteProgramadosClick(Sender: TObject);
begin
    GenerarReporteProgramados;
end;

procedure TfrmReportes.btnReporteContactosClick(Sender: TObject);
begin

```

```

    GenerarReporteContactos;
end;
procedure TfrmReportes.GenerarReporteCorreos;
var
    DotFile, PNGFile, CarpetaReportes: string;
begin
    if FUsuario.Inbox = nil then
    begin
        ShowMessage('El usuario no tiene correos.');
```

Exit;

```
    end;

    // Crear carpeta de reportes si no existe
    CarpetaReportes := LowerCase(FUsuario.user) + '-reportes';
    if not DirectoryExists(CarpetaReportes) then
        CreateDir(CarpetaReportes);

    DotFile := CarpetaReportes + PathDelim + 'Bandeja_' + FUsuario.email +
'.dot';
    PNGFile := CarpetaReportes + PathDelim + 'Bandeja_' + FUsuario.email +
'.png';

    GenerateDotBandeja(FUsuario.Inbox, DotFile, PNGFile);

    ShowMessage('Reporte de Correos Recibidos generado en: ' + PNGFile);
end;

procedure TfrmReportes.GenerarReportePapelera;
var
    DotFile, PNGFile, CarpetaReportes: string;
begin
    if (FUsuario.Papelera = nil) or (FUsuario.Papelera^.Top = nil) then
    begin
        ShowMessage('La Papelera del usuario está vacía.');
```

Exit;

```
    end;

    CarpetaReportes := LowerCase(FUsuario.user) + '-reportes';
    if not DirectoryExists(CarpetaReportes) then
        CreateDir(CarpetaReportes);

    DotFile := CarpetaReportes + PathDelim + 'Papelera_' + FUsuario.email +
'.dot';
```

```

    PNGFile := CarpetaReportes + PathDelim + 'Papelera_' + FUsuario.email +
'.png';

    GenerateDotPapelera(FUsuario.Papelera, DotFile, PNGFile);

    ShowMessage('Reporte de la Papelera generado en: ' + PNGFile);
end;

procedure TfrmReportes.GenerarReporteProgramados;
var
    DotFile, PNGFile, CarpetaReportes: string;
begin
    if (FUsuario.Programados = nil) or (FUsuario.Programados^.Top = nil) then
    begin
        ShowMessage('No hay correos programados para este usuario.');
```

Exit;

end;

```

    CarpetaReportes := LowerCase(FUsuario.user) + '-reportes';
    if not DirectoryExists(CarpetaReportes) then
        CreateDir(CarpetaReportes);

    DotFile := CarpetaReportes + PathDelim + 'Programados_' + FUsuario.email
+ '.dot';
    PNGFile := CarpetaReportes + PathDelim + 'Programados_' + FUsuario.email
+ '.png';

    GenerateDotProgramados(FUsuario.Programados, DotFile, PNGFile);

    ShowMessage('Reporte de Correos Programados generado en: ' + PNGFile);
end;

procedure TfrmReportes.GenerarReporteContactos;
var
    DotFile, PNGFile, CarpetaReportes: string;
begin
    if (FUsuario.Contacts = nil) or (FUsuario.Contacts^.Head = nil) then
    begin
        ShowMessage('El usuario no tiene contactos.');
```

Exit;

end;

```

    CarpetaReportes := LowerCase(FUsuario.user) + '-reportes';
```

```

if not DirectoryExists(CarpetaReportes) then
  CreateDir(CarpetaReportes);

  DotFile := CarpetaReportes + PathDelim + 'Contactos_' + FUsuario.email +
  '.dot';
  PNGFile := CarpetaReportes + PathDelim + 'Contactos_' + FUsuario.email +
  '.png';

  GenerateDotContactos(FUsuario.Contacts, DotFile, PNGFile);

  ShowMessage('Reporte de Contactos generado en: ' + PNGFile);
end;

end.

```

uVerContactos

Se encarga de mostrar de manera visible los contactos agregados en uContactos

```

unit uVerContactos;

{$mode delphi}{$H+}

interface

uses
  Classes, SysUtils, Forms, Controls, Graphics, Dialogs, StdCtrls,
  UContactTypes, SLL;

type

  { TfrmVerContactos }

  TfrmVerContactos = class(TForm)
    btnSiguiente: TButton;
    btnAnterior: TButton;
    lblNombre: TLabel;
    lblUsuario: TLabel;
    lblEmail: TLabel;
    lblTelefono: TLabel;
    procedure btnSiguienteClick(Sender: TObject);
    procedure btnAnteriorClick(Sender: TObject);
  end;

```



```

private
    FContactoActual: PContactNode;
    FListaContactos: PContactList;
    procedure MostrarContacto;
public
    UsuarioEmail: string;
    procedure InicializarContactos; // <<--- nuevo método
end;

var
    frmVerContactos: TfrmVerContactos;

implementation

{$R *.lfm}

procedure TfrmVerContactos.InicializarContactos;
var
    Data: TDataUser;
begin
    Caption := 'Contactos';
    Data := SSL_GETBYEMAIL(UsuarioEmail);
    if Data.Contacts <> nil then
        FListaContactos := Data.Contacts
    else
        FListaContactos := nil;

    FContactoActual := nil;
    if (FListaContactos <> nil) and (FListaContactos^.Head <> nil) then
        FContactoActual := FListaContactos^.Head;

    MostrarContacto;
end;

procedure TfrmVerContactos.MostrarContacto;
begin
    if (FContactoActual = nil) then
    begin
        lblNombre.Caption := 'Nombre: ---';
        lblUsuario.Caption := 'Usuario: ---';
        lblEmail.Caption := 'Email: ---';
        lblTelefono.Caption := 'Teléfono: ---';
        Exit;
    end;
end;

```

```

end;

lblNombre.Caption := 'Nombre: ' + FContactoActual^.name;
lblUsuario.Caption := 'Usuario: ' + FContactoActual^.user;
lblEmail.Caption := 'Email: ' + FContactoActual^.email;
lblTelefono.Caption := 'Teléfono: ' + FContactoActual^.phone;
end;

procedure TfrmVerContactos.btnSiguienteClick(Sender: TObject);
begin
    if FContactoActual <> nil then
        FContactoActual := FContactoActual^.Next;
        MostrarContacto;
    end;

procedure TfrmVerContactos.btnAnteriorClick(Sender: TObject);
var
    Actual: PContactNode;
begin
    if (FListaContactos = nil) or (FListaContactos^.Head = nil) then Exit;

    Actual := FListaContactos^.Head;
    if FContactoActual = nil then
        begin
            FContactoActual := Actual;
            MostrarContacto;
            Exit;
        end;

    // buscar el nodo anterior
    while Actual^.Next <> FContactoActual do
        begin
            Actual := Actual^.Next;
            if Actual = FListaContactos^.Head then Break; // ciclo completo
        end;
    FContactoActual := Actual;
    MostrarContacto;
end;

end.

```

uBorradores

```
unit uBorradores;

{$mode delphi}{$H+}

interface

uses
  Classes, SysUtils, Forms, Controls, Graphics, Dialogs, StdCtrls,
  SLL, UAVL, DLL_CON, UContactTypes, Process;

type
  { TfrmBorradores }
  TfrmBorradores = class(TForm)
    btnPreOrden: TButton;
    btnInOrden: TButton;
    btnPostOrden: TButton;
    btnEnviarSeleccionado: TButton;
    btnEnviarTodos: TButton;
    btnReporte: TButton;
    lstBorradores: TListBox;
    edtAsunto: TEdit;
    edtDestinatario: TEdit;
    memoMensaje: TMemo;
    lblDestinatario: TLabel;
    lblAsunto: TLabel;
    lblMensaje: TLabel;

    procedure btnPreOrdenClick(Sender: TObject);
    procedure btnInOrdenClick(Sender: TObject);
    procedure btnPostOrdenClick(Sender: TObject);
    procedure lstBorradoresClick(Sender: TObject);
    procedure btnEnviarSeleccionadoClick(Sender: TObject);
    procedure btnEnviarTodosClick(Sender: TObject);
    procedure btnReporteClick(Sender: TObject);
  private
    FUsuario: TDataUser;
    procedure MostrarRecorrido(orden: string);
    procedure CargarCorreoSeleccionado(const id: string);
    procedure RecorrerPreOrden(nodo: PNodoAVL; list: TStringList);
    procedure RecorrerInOrden(nodo: PNodoAVL; list: TStringList);
    procedure RecorrerPostOrden(nodo: PNodoAVL; list: TStringList);
```

```

    public
        procedure SetUsuarioEmail(const email: string);
    end;

var
    frmBorradores: TfrmBorradores;

implementation

{$R *.lfm}

{ TfrmBorradores }

procedure TfrmBorradores.SetUsuarioEmail(const email: string);
begin
    FUsuario := SSL_GETBYEMAIL(email);
end;

procedure TfrmBorradores.RecorrerPreOrden(nodo: PNodeAVL; list:
TStringList);
begin
    if nodo = nil then Exit;
    list.Add(nodo^.mail.id);
    RecorrerPreOrden(nodo^.izquierda, list);
    RecorrerPreOrden(nodo^.derecha, list);
end;

procedure TfrmBorradores.RecorrerInOrden(nodo: PNodeAVL; list:
TStringList);
begin
    if nodo = nil then Exit;
    RecorrerInOrden(nodo^.izquierda, list);
    list.Add(nodo^.mail.id);
    RecorrerInOrden(nodo^.derecha, list);
end;

procedure TfrmBorradores.RecorrerPostOrden(nodo: PNodeAVL; list:
TStringList);
begin
    if nodo = nil then Exit;
    RecorrerPostOrden(nodo^.izquierda, list);
    RecorrerPostOrden(nodo^.derecha, list);
    list.Add(nodo^.mail.id);
end;

```

```

end;

procedure TfrmBorradores.MostrarRecorrido(ordena: string);
var
  lista: TStringList;
begin
  lstBorradores.Clear;
  lista := TStringList.Create;
  try
    if (FUsuario.Drafts <> nil) and (FUsuario.Drafts^.raiz <> nil) then
    begin
      if ordena = 'pre' then
        RecorrerPreOrden(FUsuario.Drafts^.raiz, lista)
      else if ordena = 'in' then
        RecorrerInOrden(FUsuario.Drafts^.raiz, lista)
      else if ordena = 'post' then
        RecorrerPostOrden(FUsuario.Drafts^.raiz, lista);

      lstBorradores.Items.Assign(lista);
    end
  else
    ShowMessage('No hay borradores guardados.');
```

```

  finally
    lista.Free;
  end;
end;

procedure TfrmBorradores.CargarCorreoSeleccionado(const id: string);
var
  nodo: PNodeAVL;
  correo: TMail;
begin
  if (FUsuario.Drafts = nil) or (FUsuario.Drafts^.raiz = nil) then Exit;

  nodo := BuscarAVL(FUsuario.Drafts^.raiz, id);
  if nodo <> nil then
  begin
    correo := nodo^.mail;
    edtDestinatario.Text := correo.destinatario;
    edtAsunto.Text := correo.asunto;
    memoMensaje.Text := correo.mensaje;
  end
  else

```

```

        ShowMessage('No se encontró el borrador seleccionado.');
```

end;

```

procedure TfrmBorradores.lstBorradoresClick(Sender: TObject);
var
    seleccion: string;
begin
    if lstBorradores.ItemIndex <> -1 then
    begin
        seleccion := lstBorradores.Items[lstBorradores.ItemIndex];
        CargarCorreoSeleccionado(seleccion);
    end;
end;

procedure TfrmBorradores.btnPreOrdenClick(Sender: TObject);
begin
    MostrarRecorrido('pre');
end;

procedure TfrmBorradores.btnInOrdenClick(Sender: TObject);
begin
    MostrarRecorrido('in');
end;

procedure TfrmBorradores.btnPostOrdenClick(Sender: TObject);
begin
    MostrarRecorrido('post');
end;

procedure TfrmBorradores.btnEnviarSeleccionadoClick(Sender: TObject);
var
    idCorreo: string;
    nodo: PNodoAVL;
    correo: TMail;
    inboxDestinatario: PMsgList;
begin
    if lstBorradores.ItemIndex = -1 then
    begin
        ShowMessage('Seleccione un borrador primero.');
```

Exit;

end;

```

    idCorreo := lstBorradores.Items[lstBorradores.ItemIndex];
```

```

nodo := BuscarAVL(FUsuario.Drafts^.raiz, idCorreo);
if nodo = nil then
begin
    ShowMessage('No se encontró el borrador seleccionado.');
```

Exit;

```
end;
```

correo := nodo^.mail;

```
if Trim(correo.destinatario) = '' then
begin
    ShowMessage('El borrador no tiene destinatario.');
```

Exit;

```
end;
```

inboxDestinatario := SSL_GETBYEMAIL(correo.destinatario).Inbox;

```
if inboxDestinatario = nil then
begin
    ShowMessage('No se pudo acceder a la bandeja de entrada del
destinatario.');
```

Exit;

```
end;
```

DLL_Insert(inboxDestinatario,

correo.id,

correo.remitente,

correo.asunto,

correo.mensaje,

FormatDateTime('dd/mm/yyyy hh:nn:ss', Now));

FUsuario.Drafts^.raiz := EliminarAVL(FUsuario.Drafts^.raiz, correo.id);

ShowMessage('Correo enviado y eliminado de borradores.');

MostrarRecorrido('in');

```
end;
```

```
procedure TfrmBorradores.btnEnviarTodosClick(Sender: TObject);
var
    enviados: TStringList;
```

procedure RecorrerYEnviar(nodo: PNodeAVL);

```
var
    inboxDestinatario: PMsgList;
```

```

    correo: TMail;
begin
    if nodo = nil then Exit;

    RecorrerYEnviar(nodo^.izquierda);

    correo := nodo^.mail;
    if Trim(correo.destinatario) <> '' then
    begin
        inboxDestinatario := SSL_GETBYEMAIL(correo.destinatario).Inbox;
        if inboxDestinatario <> nil then
        begin
            DLL_Insert(inboxDestinatario,
                        correo.id,
                        correo.remitente,
                        correo.asunto,
                        correo.mensaje,
                        FormatDateTime('dd/mm/yyyy hh:nn:ss', Now));
            enviados.Add(correo.id);
        end;
    end;

    RecorrerYEnviar(nodo^.derecha);
end;

var
    i: Integer;
begin
    if (FUsuario.Drafts = nil) or (FUsuario.Drafts^.raiz = nil) then
    begin
        ShowMessage('No hay correos en borradores.');
```

Exit;

```
    end;

    enviados := TStringList.Create;
    try
        RecorrerYEnviar(FUsuario.Drafts^.raiz);
        for i := 0 to enviados.Count - 1 do
            FUsuario.Drafts^.raiz := EliminarAVL(FUsuario.Drafts^.raiz,
enviados[i]);
        finally
            enviados.Free;
        end;
    end;
end;

```



```

    ShowMessage('Todos los correos en borradores fueron enviados y
eliminados.');
```

```

    MostrarRecorrido('in');
```

```

end;
```

```

// ===== NUEVO: GENERAR REPORTE =====
procedure TfrmBorradores.btnReporteClick(Sender: TObject);
var
    dotFile, pngFile: string;
    Output: AnsiString;
begin
    if (FUsuario.Drafts = nil) or (FUsuario.Drafts^.raiz = nil) then
    begin
        ShowMessage('No hay borradores para graficar.');
```

```

        Exit;
```

```

    end;
```

```

    if not DirectoryExists('Reportes') then
        CreateDir('Reportes');
```

```

    dotFile := 'Reportes/Borradores.dot';
    pngFile := 'Reportes/Borradores.png';
```

```

    GenerarDOTAVL(FUsuario.Drafts^.raiz, dotFile);
```

```

    if FileExists('/usr/bin/dot') then
    begin
        RunCommand('/usr/bin/dot', ['-Tpng', dotFile, '-o', pngFile], Output);
        ShowMessage('Reporte generado en: ' + pngFile);
    end
    else
        ShowMessage('Graphviz no está instalado o no se encuentra el ejecutable
"dot".');
```

```

end;
```

```

end.
```

uFavoritos

```

unit uFavoritos;
```

```

{$mode delphi}{$H+}

interface

uses
  Classes, SysUtils, Forms, Controls, Graphics, Dialogs, StdCtrls, Grids,
  SLL, UBTTree, Process; // UBTTree: TMail, PNodeB, BuscarB, InOrdenBLista,
  EliminarBTree

type
  { TfrmFavoritos }
  TfrmFavoritos = class(TForm)
    lblTotal: TLabel;
    lblDetalle: TLabel;
    lblRemitente: TLabel;
    lblAsunto: TLabel;
    lblFecha: TLabel;
    memoMensaje: TMemo;
    gridFavoritos: TStringGrid;
    btnEliminar: TButton;
    btnReporte: TButton;

    procedure FormShow(Sender: TObject);
    procedure gridFavoritosSelectCell(Sender: TObject; aCol, aRow: Integer;
      var CanSelect: Boolean);
    procedure btnEliminarClick(Sender: TObject);
    procedure btnReporteClick(Sender: TObject);

  private
    FUsuario: TDataUser;
    FCorreoSeleccionado: string; // ID del correo seleccionado
    procedure CargarFavoritos;
    procedure MostrarDetalles(const id: string);
    procedure LimpiarDetalles;
  public
    procedure SetUsuarioEmail(const email: string);
  end;

var
  frmFavoritos: TfrmFavoritos;

implementation

```

```

{$R *.lfm}

{ TfrmFavoritos }

procedure TfrmFavoritos.SetUsuarioEmail(const email: string);
begin
    FUsuario := SSL_GETBYEMAIL(email);
end;

procedure TfrmFavoritos.CargarFavoritos;
var
    lista: TStringList;
    i: Integer;
    partes: TStringArray;
begin
    gridFavoritos.RowCount := 1;
    lista := TStringList.Create;
    try
        if (FUsuario.Favoritos <> nil) and (FUsuario.Favoritos^.raiz <> nil)
    then
        begin
            InOrdenBLista(FUsuario.Favoritos^.raiz, lista);

            gridFavoritos.RowCount := lista.Count + 1;
            for i := 0 to lista.Count - 1 do
                begin
                    partes := lista[i].Split(['|']);
                    if Length(partes) >= 3 then
                        begin
                            gridFavoritos.Cells[0, i+1] := partes[0];
                            gridFavoritos.Cells[1, i+1] := partes[1];
                            gridFavoritos.Cells[2, i+1] := partes[2];
                        end;
                    end;

                    lblTotal.Caption := 'Total favoritos: ' + IntToStr(lista.Count);
                end
            else
                lblTotal.Caption := 'Total favoritos: 0';
            finally
                lista.Free;
            end;
        end;
    end;
end;

```

```

end;

procedure TfrmFavoritos.MostrarDetalles(const id: string);
var
    correo: TMail;
begin
    correo := BuscarB(FUsuario.Favoritos^.raiz, id);
    if correo.id <> '' then
    begin
        lblRemitente.Caption := 'Remitente: ' + correo.remitente;
        lblAsunto.Caption := 'Asunto: ' + correo.asunto;
        lblFecha.Caption := 'Fecha: '; // puedes ajustar si tu TMail incluye
        fecha
        memoMensaje.Text := correo.mensaje;
        FCorreoSeleccionado := correo.id;
    end;
end;

procedure TfrmFavoritos.LimpiarDetalles;
begin
    lblRemitente.Caption := 'Remitente: ';
    lblAsunto.Caption := 'Asunto: ';
    lblFecha.Caption := 'Fecha: ';
    memoMensaje.Clear;
    FCorreoSeleccionado := '';
end;

procedure TfrmFavoritos.FormShow(Sender: TObject);
begin
    gridFavoritos.ColCount := 3;
    gridFavoritos.RowCount := 1;
    gridFavoritos.FixedRows := 1;
    gridFavoritos.Cells[0,0] := 'ID';
    gridFavoritos.Cells[1,0] := 'Asunto';
    gridFavoritos.Cells[2,0] := 'Remitente';

    CargarFavoritos;
    LimpiarDetalles;
end;

procedure TfrmFavoritos.gridFavoritosSelectCell(Sender: TObject; aCol,
aRow: Integer;
    var CanSelect: Boolean);

```

```

var
  id: string;
begin
  if aRow > 0 then
  begin
    id := gridFavoritos.Cells[0,aRow];
    if id <> '' then
      MostrarDetalles(id);
    end;
  end;
end;

procedure TfrmFavoritos.btnEliminarClick(Sender: TObject);
begin
  if FCorreoSeleccionado = '' then
  begin
    ShowMessage('Seleccione un correo primero.');
```

Exit;

end;

EliminarBTree(FUsuario.Favoritos^.raiz, FCorreoSeleccionado);

ShowMessage('Correo eliminado de favoritos.');

LimpiarDetalles;

CargarFavoritos;

end;

// ===== NUEVO: GENERAR REPORTE =====

```

procedure TfrmFavoritos.btnReporteClick(Sender: TObject);
var
  dotFile, pngFile: string;
  Output: AnsiString;
begin
  if (FUsuario.Favoritos = nil) or (FUsuario.Favoritos^.raiz = nil) then
  begin
    ShowMessage('No hay favoritos para graficar.');
```

Exit;

end;

if not DirectoryExists('Reportes') then

CreateDir('Reportes');

dotFile := 'Reportes/Favoritos.dot';

pngFile := 'Reportes/Favoritos.png';

```

GenerarDOTB(FUsuario.Favoritos^.raiz, dotFile);

if FileExists('/usr/bin/dot') then
begin
    RunCommand('/usr/bin/dot', ['-Tpng', dotFile, '-o', pngFile], Output);
    ShowMessage('Reporte generado en: ' + pngFile);
end
else
    ShowMessage('Graphviz no está instalado o no se encuentra el ejecutable
"dot".');
end;

end.

```

uCrearComunidadesArbol

```

unit uCrearComunidadesArbol;

{$mode delphi}

interface

uses
    Classes, SysUtils, Forms, Controls, Graphics, Dialogs, StdCtrls,
    BSTCOMUNIDADES, Process;

type
    { TfrmCrearComunidadesArbol }
    TfrmCrearComunidadesArbol = class(TForm)
        lblTitulo: TLabel;
        lblNombre: TLabel;
        lblComunidad: TLabel;
        lblCorreo: TLabel;
        lblMensaje: TLabel;
        lblFecha: TLabel;
        txtNombre: TEdit;
        txtComunidad: TEdit;
        txtCorreo: TEdit;
    end;

```

```

    txtMensaje: TEdit;
    txtFecha: TEdit;
    btnCrear: TButton;
    btnAgregar: TButton;
    btnInsertarMsg: TButton;
    btnCerrar: TButton;
    btnReporte: TButton;
    procedure btnCrearClick(Sender: TObject);
    procedure btnAgregarClick(Sender: TObject);
    procedure btnInsertarMsgClick(Sender: TObject);
    procedure btnCerrarClick(Sender: TObject);
    procedure btnReporteClick(Sender: TObject);
  private
  public
  end;

var
  frmCrearComunidadesArbol: TfrmCrearComunidadesArbol;

implementation

{$R *.lfm}

procedure TfrmCrearComunidadesArbol.btnCrearClick(Sender: TObject);
var
  response: Boolean;
begin
  response := InsertBSTNode(txtNombre.Text, DateToStr(Now));
  if response then
    ShowMessage('Comunidad creada correctamente.')
  else
    ShowMessage('Error: Ya existe una comunidad con ese nombre.');
```

```

end;

procedure TfrmCrearComunidadesArbol.btnAgregarClick(Sender: TObject);
var
  response: Boolean;
begin
  response := insertUsers(txtComunidad.Text, txtCorreo.Text);
  if response then
    ShowMessage('Usuario agregado correctamente a la comunidad.')
  else
    ShowMessage('Error: Verifica que la comunidad exista o que el usuario
```

```

ya esté en la comunidad.');
```

```

end;

procedure TfrmCrearComunidadesArbol.btnInsertarMsgClick(Sender: TObject);
var
    response: Boolean;
begin
    response := insertEmails(txtComunidad.Text, txtCorreo.Text,
txtMensaje.Text, txtFecha.Text);
    if response then
        ShowMessage('Mensaje agregado correctamente a la comunidad.')
    else
        ShowMessage('Error: Verifica que la comunidad y usuario existan en la
comunidad.');
```

```

end;

procedure TfrmCrearComunidadesArbol.btnCerrarClick(Sender: TObject);
begin
    Close;
end;

procedure TfrmCrearComunidadesArbol.btnReporteClick(Sender: TObject);
var
    dotFile, pngFile: string;
    dotCode: string;
    SL: TStringList;
    Output: string;
begin
    dotFile := 'reportes/comunidades.dot';
    pngFile := 'reportes/comunidades.png';

    // Generamos el DOT desde el BST
    dotCode := BST_GENERARDOT();

    // Guardamos el DOT
    SL := TStringList.Create;
    try
        SL.Text := dotCode;
        if not DirectoryExists('reportes') then
            CreateDir('reportes');
        SL.SaveToFile(dotFile);
    finally
        SL.Free;
    end;
end;

```



```
end;

// Ejecutamos Graphviz para crear el PNG
if FileExists('/usr/bin/dot') then
begin
    RunCommand('/usr/bin/dot', ['-Tpng', dotFile, '-o', pngFile], Output);
    ShowMessage('Reporte generado en: ' + pngFile);
end
else
    ShowMessage('No se encontró Graphviz (dot). Instálalo para generar el gráfico.');
```

end;

end.